Contents

Contents i

List of Figures iii

List of Tables iv

I Introduction 1

II What is TAMPINES 2

0.1 Equations of State 3

IIISteam Tables 4

IVRankine Cycle Derivations 5

1 Mass, Energy and Momentum Balance 7

1.1 Raw forms of mass, momentum and energy balance 7
1.2 Boundary Conditions 7
1.3 Steam Turbine Modelling 8
1.4 Converting forms to Mass Flowrate 27
1.5 Approach to Solving nonlinear Equations 28
1.6 PISO Algorithm 31
1.7 SIMPLE Algorithm 33
1.8 PIMPLE Algorithm 35
1.9 Application to Rankine Cycle 35

2 rhoPimpleFoam in OpenFOAM 38

3 Translation for rhoPimpleFoam to 1D case 54

V Appendix: Incompressible Equations **63**

Bibliography **65**

List of Figures

1.1 Nozzle Diagram 15
1.2 Force Balance Diagram 20

List of Tables

Acknowledgments

Part I

Introduction

Part II

What is TAMPINES

0.1 Equations of State

For helium, PR-EOS requires some quantum correction [Aasen et al., 2020].

Part III

Steam Tables

Part IV

Rankine Cycle Derivations

In this part, we consider a four control volume system representing a simple Rankine cycle. We will adapt the rhoPimpleFoam algorithm from OpenFOAM in order to solve the multiphase equations for the Rankine Cycle. This can just as easily be done for the Brayton Cycle. Do note, we are using the Homogeneous Equation Model, the simplest of them for two phase flow described in 12.5.1 of Kazimi Vol ii [Todreas, Kazimi, and Massoud, 2021]. This is much simpler than MELCOR, which was used for AP-1000 PWR simulations [Malicki et al., 2019] and uses the two-fluid two phase model for its thermal hydraulics Kim et al., 2014 wherein separate equations are solved for the two velocities under two phase flow, section 1.3 [Gauntt et al., 2000].

This will form the theoretical basis to make a solver for transient balances in this Rankine Cycle.

Chapter 1

Mass, Energy and Momentum Balance

In OpenFOAM, the PIMPLE algorithm is often used to solve for incompressible fluid flow [Devolder et al., 2015; Greenshields and Weller, 2022], this is one of the many standard solvers for solving flow in OpenFOAM [Penttinen, Yasari, and Nilsson, 2011]. Jasak has published this in his PhD thesis, available online, [Jasak, 1996] on page 143-149.

1.1 Raw forms of mass, momentum and energy balance

Let's consider mass balance and momentum balance:

$$\frac{\partial m}{\partial t} = \sum_i m_i \tag{1.1}$$

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.2}$$

$\nabla \cdot \tau$ represents shear, ρg represents the gravity body force, $-\nabla P$ represents the pressure gradient.

$$V_j \frac{\partial \rho_i h_i}{\partial t} + \sum_j \dot{m}_j h_j = Q_i - W_i + \sum_j \tau_j \cdot \mathbf{u}_j \tag{1.3}$$

Where $+\sum_j \tau_j \cdot \mathbf{u}_j$ is basically the friction loss terms where energy is transformed from kinetic energy to sensible heat. Q and W represent the thermodynamic work.

1.2 Boundary Conditions

For such flow in a Rankine cycle, we need to consider boundary conditions. The main ones are those of pressure, enthalpy and velocity, as each will be solved.

For mass balance, the boundary conditions are clearly cyclic as steam flows in a cycle, unless it is leaked, or such. For leakage, a source term or loss term can be added volumetrically or at the surface. It doesn't really matter much for a 1D case.

For momentum balance, we do not care too much about shockwaves unless we do detailed modelling of turbines and pumps. In the momentum balance, the boundary conditions for pressure need to be considered. In this regard, there are two main places where pressure is set. That is, before and after the pump. There is a positive pressure gradient and a positive flow because of a pressure source term.

For simplistic modelling, we can consider using a fixed pressure increase so that the condenser pressure is set, and a fixed pressure increase is set at a constant flowrate.

1.3 Steam Turbine Modelling

In the turbine, we can simply model it as having a fixed pressure drop [Zimmer, 2008]. However, I want something more substantial. From first principles, steam turbines extract energy due to momentum transfer from steam to the blades as it is for impulse turbines [Kareem et al., 2018]. Dixon et al. writes extensively about this in his textbook [Dixon and Hall, 2013].

The key is this:

$$W = \dot{m}\Delta h$$

How then do we model the enthalpy change? It is very much related to turbine RPM and so forth. Let us investigate the models. Now, for power plants, the most common type is the axial turbine [Agromayor and Nord, 2019], and to calculate the energy balances across here, we correlate enthalpy changes using stagnation properties [Agromayor and Nord, 2019]. Using Eq 17 of Agromayor [Agromayor and Nord, 2019]:

$$h_{in} + 0.5v_{in}^2 = h_{out} + 0.5v_{out}^2$$

In steam turbines specifically, kinetic energy cannot be neglected in the energy balance. Some of this kinetic energy is transferred to the rotor via momentum balance. To understand how much momentum is transferred, we need to understand the inner workings of steam turbines.

Now, how can we break down steam turbines to something more bite sized and understandable? We can simplify them as they are considered a series of Rotor and Stator blades [Dixon and Hall, 2013]. This is known as a cascade model under section 2.4 of Agromayor et al.'s paper [Agromayor and Nord, 2019]. Stator blades help guide steam into the Rotor blades, and they are stationary to help guide flow. A simple Darcy Weisbach equation may be sufficient to model losses over the stator blades. Otherwise, we may use some other empirical loss correlations.

What I want to do here is to correlate the enthalpy change to a turbine rotation speed.

Angular Momentum Equation

For rotational kinetic energy, or angular momentum, we can do the following balance:

$$\frac{\partial(I\omega)}{\partial t} = \text{source} - \text{sinks}$$

Of course, moment of inertia for the turbine is constant:

$$I\frac{\partial\omega}{\partial t} = \text{source} - \text{sinks}$$

Euler's Turbine Equation

Here, we can use Euler's Turbine Equation (equation 1.4 in order to determine the source terms):

$$\Delta h = \omega(r_c v_c - r_b v_b) \tag{1.4}$$

where, c and b indicate properties of the turbine at the beginning and end of the stage. v is the steam velocity

What then determines this velocity change? Now, we come back to mass, energy and momentum balance of the fluid. For energy and momentum balance, we use conservation of rotalphy [Agromayor and Nord, 2019]. Mass balance can be a simple continuity equation, though technically, it should be a transient balance. Nevertheless, that will govern the velocity at the exit.

This will be a custom energy sink in the turbine stage. We can do a single turbine stage for starters.

Regenerative Braking via Faraday's Law

As for the sinks of the turbine angular momentum balance, we need to account for friction, and electromotive force. Now, of course, we can be rather detailed in these two things, but for simplicity, friction and the regenerative braking effect can be assumed to be proportional to ω. For regenerative braking, we can use [Zhang et al., 2022]:

$$E_{regen} = \eta \int F_{brake} v dt$$

Where η can be taken to be 0.85-0.94 [Zhang et al., 2022]. We can back calculate the braking force using the energy balance:

$$P = \frac{\varepsilon^2}{R}$$

Let's differentiate the equations for regen braking:

$$P_{regen} = \eta F_{brake} v$$

$$\frac{\varepsilon^2}{R} = \eta F_{brake} v$$

For a coiled solenoid in a magnetic field, the flux linkage Phi is:

$$\Phi = NBA \sin\theta$$

We take that $\theta = \omega t$. So if we differentiate things using Faraday's law and Lenz's law:

$$\varepsilon = -\frac{d}{dt}\Phi = -NBA\omega(\cos\omega t)$$

Substituting in:

$$\frac{(-NBA\omega(\cos\omega t))^2}{R} = \eta F_{brake} v$$

$$\frac{(N^2B^2A^2\omega^2(\cos^2\omega t))}{R} = \eta F_{brake} v$$

$$\frac{(N^2B^2A^2)}{R}\omega^2(\cos^2\omega t) = \eta F_{brake} v$$

We note that for the velocity of the car, v, this is related to the speed of the "wheel" of the car. In the turbine terms, this v is the shaft radius times ω.

$$\frac{(N^2B^2A^2)}{R}\omega^2(\cos^2\omega t) = \eta F_{brake} r_{shaft}\omega$$

$$\frac{(N^2B^2A^2)}{\eta R_{load} r_{shaft}}\omega(\cos^2\omega t) = F_{brake}$$

If we have multiple coils, then we can have various coils at various phase omegas

$$\frac{(N^2B^2A^2)}{\eta R_{load} r_{shaft}}\omega \sum_j \cos^2(\omega t + b_j) = F_{brake}$$

b_j in this case is a phase shift, if we have j coils in this generator.
In the case that the load is disconnected $R_{load} \to \infty$ and the braking force tends to zero.
With braking force, we can find a braking torque using $\tau = F r_{shaft}$:

$$I\frac{\partial\omega}{\partial t} = \text{source} - F_{brake} r_{shaft}$$

$$I\frac{\partial\omega}{\partial t} = \text{source} - \frac{(N^2B^2A^2)}{\eta R_{load} r_{shaft}}\omega\sum_j \cos^2(\omega t + b_j) r_{shaft}$$

So we arrive at:

$$I\frac{\partial\omega}{\partial t} = \text{source} - \frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j)$$

Discretisation Should we want to implement this code, we need to perform some time discretisation:

$$I\frac{\partial\omega}{\partial t} = \text{source} - \frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j)$$

With a semi implicit manner of discretisation:

$$I\frac{\omega^{t+\Delta t} - \omega^t}{\Delta t} = \text{source} - \frac{(N^2B^2A^2)}{\eta R_{load}}\omega^{t+\Delta t}\sum_j \cos^2(\omega^t t + b_j)$$

$$I\frac{\omega^{t+\Delta t}}{\Delta t} + \frac{(N^2B^2A^2)}{\eta R_{load}}\omega^{t+\Delta t}\sum_j \cos^2(\omega^t t + b_j) = I\frac{\omega^t}{\Delta t} + \text{source}$$

$$\omega^{t+\Delta t}\left(\frac{I}{\Delta t} + \frac{(N^2B^2A^2)}{\eta R_{load}}\sum_j \cos^2(\omega^t t + b_j)\right) = I\frac{\omega^t}{\Delta t} + \text{source}$$

Impulse Turbine

For the torque, we want to consider whether it is an impulse vs a reaction turbine. Though for steam turbines, we will have impulse stages in the beginning and reaction turbine towards the end as shown here. It is easier to think of impulse as in hitting on the blade itself, whereas reaction is treating turbine blades like airfoils.

And the torque for an impulse turbine, which is a suitable model, is as found here:

$$\tau = \dot{m}(r_c v_c - r_b v_b)$$

Substituting in:

$$I\frac{\partial\omega}{\partial t} = \dot{m}(r_c v_c - r_b v_b) - \frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j)$$

Note that v_c and v_b are tangential velocities of the steam. How then can we determine the relative velocities? This is done through velocity triangles.

Now, if we were to use velocity triangles for an **impluse** turbine, we can do some derivations as was done in NPTEL, to obtain:

$$\frac{\partial p_{tangential}}{\partial t} = \dot{m}(v_{inlet}\cos\alpha_{inlet} - v_{blade})(1+k_b)$$

Where v_{inlet} is inlet velocity (denoted v_b previously). α_{inlet} is the angle in radians for which the steam leaves the nozzle with reference to the z axis (axial or z direction of flow). The torque is then:

$$r\frac{\partial p_{tangential}}{\partial t} = \dot{m}r(v_{inlet}\cos\alpha_{inlet} - v_{blade})(1+k_b)$$

Hence, in terms of v_{inlet}, α_{inlet} and v_{blade},

$$I\frac{\partial \omega}{\partial t} = \dot{m}r(v_{inlet}\cos\alpha_{inlet} - v_{blade})(1+k_b) - \frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j)$$

But we also need to know that $v_{blade} = r_{blade}\omega$:

$$I\frac{\partial \omega}{\partial t} = \dot{m}r(v_{inlet}\cos\alpha_{inlet} - r_{blade}\omega)(1+k_b) - \frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j)$$

For multistage turbine, under constant α_{inlet} for simplicity:

$$I\frac{\partial \omega}{\partial t} = -\frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j) + \sum_j \dot{m}r_{blade}(v_{inlet}\cos\alpha_{inlet} - r_{blade}\omega)(1+k_b)$$

Suppose we can have a linear increase of r_{blade} for multistage stage turbine, and each stage constitutes a 5% expansion:

Then:

$$r_{blade} = r_{first-blade} * (1.05)^j$$

$$\begin{aligned} I\frac{\partial \omega}{\partial t} &= -\frac{(N^2B^2A^2)}{\eta R_{load}}\omega\sum_j \cos^2(\omega t + b_j) \\ &+ \sum_j 1.05^j\dot{m}r_{first-blade}(v_{inlet,j}\cos\alpha_{inlet} - 1.05^j r_{first-blade}\omega)(1+k_b) \end{aligned}$$

Now, for $v_{inlet,j}$, we need to consider the nozzle effects.

We can use this approximation, if there is little kinetic energy at the first stage:

$$v_{inlet,1}^2 \approx \sqrt{2|\Delta h|_1}$$

For nozzles in each stage, we can assume ideality and assume it is isentropic. Thus, we can use steam tables where the (h, s) functions are programmed for the steam turbine. Staging is done so that we don't have excessive turbine RPM, and that the steam velocity is also not excessively high.

Now, suppose we have $k_b = 0.75$ as is suggested here. Note that these are typical values. We can also design α_{inlet} such that it is about 70 degrees.

We note then that $v_{outlet,1} = 0.75 v_{inlet,1}$. We then need to expand the flow.

Noting conservation of energy demands that:

$$\dot{m}_1 \left(h + 0.5v^2\right)_1 = \dot{m}_2 \left(h + 0.5v^2\right)_2$$

With conservation of mass, $\dot{m}_1 = \dot{m}_2$:

$$\left(h + 0.5v^2\right)_1 = \left(h + 0.5v^2\right)_2$$

We can take a look at the mass balance equations:

$$\rho_1 A_1 u_1 = \rho_2 A_2 u_2$$

For a nozzle, $A_2 < A_1$, that is, the exit area is lower than the entry area.

$$\frac{A_1}{A_2} \rho_1 u_1 = \rho_2 u_2$$

We get:

$$u_2 = \frac{A_1 \rho_1}{A_2 \rho_2} u_1$$

$$h_1 + 0.5v_1^2 = h_2 + 0.5 \left(\frac{A_1 \rho_1}{A_2 \rho_2}\right)^2 v_1^2$$

$$h_1 = h_2 + 0.5 \left(\left(\frac{A_1 \rho_1}{A_2 \rho_2}\right)^2 - 1\right) v_1^2$$

So we arrive at:

$$h_1 - h_2 = 0.5 \left(\left(\frac{A_1 \rho_1}{A_2 \rho_2}\right)^2 - 1\right) v_1^2$$

(ρ, h) will be dependent on a specific thermodynamic state.

Now, once we have fixed an enthalpy difference, we can assume isentropic expansion (not isobaric, since this is strictly speaking, a turbine where pressure is being lowered). For this, we can use (p,s) or (h,s) backward equations for the steam tables. These will be low quality steam for which (h, s) tables are ideal.

Now, is there some geometric mechanism which determines the change in enthalpy?

In an incompressible flow, or where density change is small,

$$h_1 - h_2 = 0.5\left(\left(\frac{A_1}{A_2}\right)^2 - 1\right)v_1^2$$

The exit enthalpy is fully determined. However, we need one more equation in order to fully specify this for compressible flow, and it is not just human set.

Of course, in wet steam turbines, such as in nuclear applications, we expect steam to be saturated and condensing during expansion. But this may not be the case as there is supersaturation in metastable steam because the steam expands so quickly there is no time for vapour formation.

But I digress, there still needs to be a way to determine the thermodynamic state of steam at the outlet of the nozzle. How is this done?

We still have conservation of momentum. Assuming the ideal state, where no momentum is lost in the nozzle (isentropic anyway).

$$\sum F_{CV} = \frac{D}{Dt}(mv)_{CV}$$

In a 1D case:

$$\sum F_{CV} = \frac{\partial}{\partial t}(mv)_{CV} + u\frac{\partial}{\partial x}(mv)_{CV}$$

Note, the force on the control volume is the sum of the force contributions due to pressure

$$P_2A_2 - P_1A_1 = \sum F_{CV}$$

Substituting in,

$$P_2A_2 - P_1A_1 = u\frac{\partial}{\partial x}(mv)_{CV}$$

$$P_2A_2 - P_1A_1 = u\frac{\partial}{\partial x}(\rho A_{XS}\Delta x v)_{CV}$$

$$P_2A_2 - P_1A_1 = \frac{\rho_1A_1v_1^2 - \rho_2A_2v_2^2}{\Delta x}(\Delta x)_{CV}$$

$$P_2A_2 - P_1A_1 = \rho_1A_1v_1^2 - \rho_2A_2v_2^2$$

In terms of mass flowrate, $\dot{m} = \rho_1A_1v_1 = \rho_2A_2v_2$:

$$\frac{P_2A_2 - P_1A_1}{\dot{m}} = v_1 - v_2$$

We note that in nozzles, $P_1 > P_2$, $A_1 > A_2$, $v_2 > v_1$ as seen in figure 1.1:

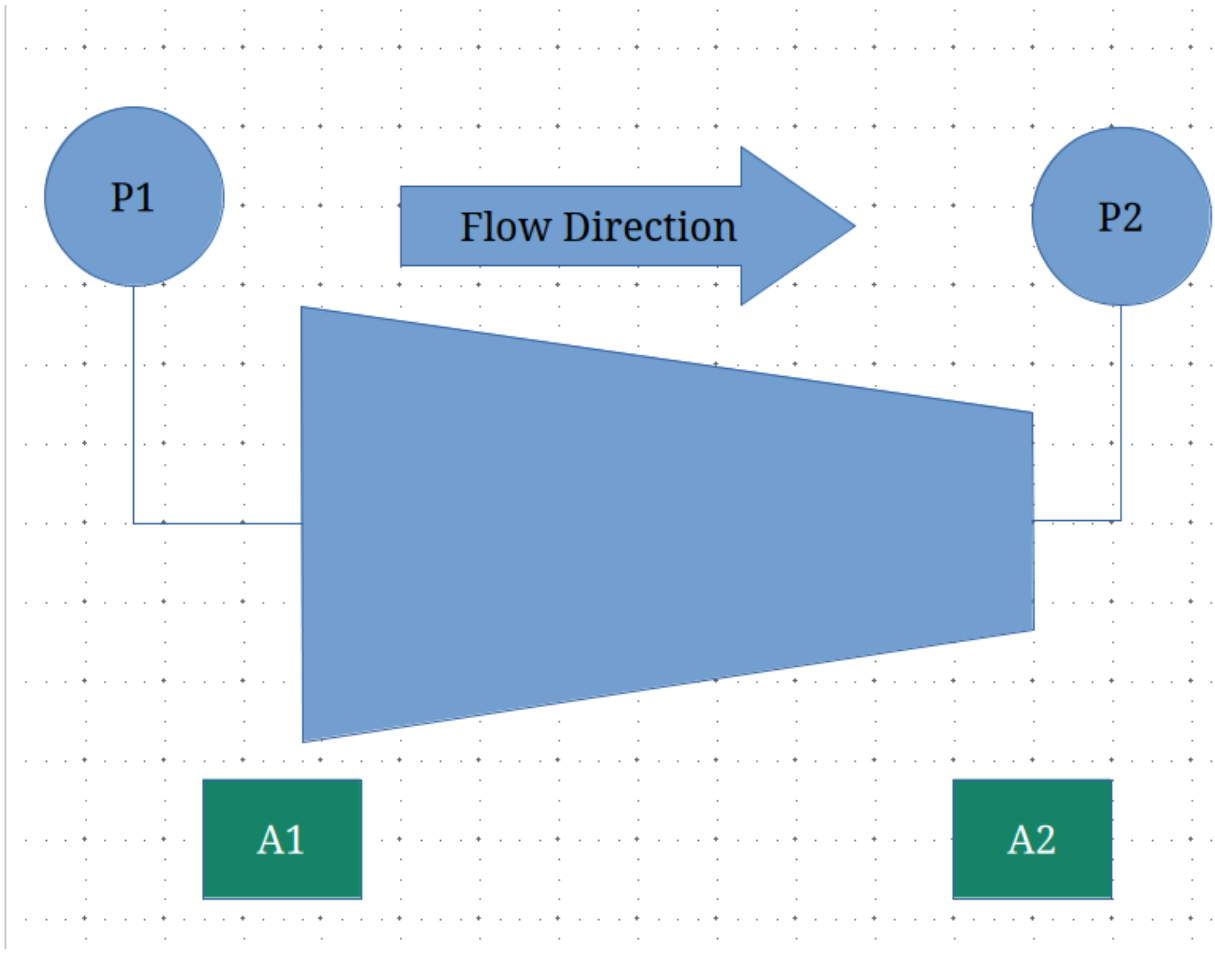

Figure 1.1: Nozzle Diagram

So the following equation should be correct:

$$\frac{P_1 A_1 - P_2 A_2}{\dot{m}} = v_2 - v_1 \tag{1.5}$$

Just want to be clear on sign convention, forces towards the right are positive, and momentum flows towards the right (larger x) are positive.

From equation 1.5, we combine this with the energy (enthalpy) balance to obtain:

$$h_1 - h_2 = 0.5 \left(\left(\frac{A_1 \rho_1}{A_2 \rho_2} \right)^2 - 1 \right) v_1^2$$

Of course, if we want to eliminate v_2, so we can find the final thermodynamic state,

$$\rho_1 A_1 v_1 = \rho_2 A_2 v_2$$

$$v_2 = \frac{\rho_1 A_1 v_1}{\rho_2 A_2}$$

$$\frac{P_1 A_1 - P_2 A_2}{\dot{m}} = \frac{\rho_1 A_1 v_1}{\rho_2 A_2} - v_1$$

$$\frac{P_1 A_1 - P_2 A_2}{\dot{m}} = v_1 \left(\frac{\rho_1 A_1}{\rho_2 A_2} - 1 \right)$$

Basically, this leaves us with two equations with which to obtain the thermodynamic state at the nozzle exit. There are three unknowns, h_2, P_2 and ρ_2 and two equations. We presume that areas are known. The steam tables provide the equation of state (EOS) which helps to provide the third equation.

How can we iterate our way there? Using these equations:

$$h_1 - h_2 = 0.5 \left(\left(\frac{\rho_1 A_1}{\rho_2 A_2} \right)^2 - 1 \right) v_1^2$$

$$\frac{P_1 A_1 - P_2 A_2}{\dot{m}} = \left(\frac{\rho_1 A_1}{\rho_2 A_2} - 1 \right) v_1$$

We could start with an initial guess of $\rho_2 = \rho_1$. From both equations, we would get h_2 and P_2 separately. Then we could get a second guess of ρ_2 using the steam tables. We calculate a residual and compare it to tolerance, and then we iterate our way till ρ_2 converges. Based on that, we have or (p_2, h_2) solution for the outlet thermodynamic state.

From there, it should be easy to calculate ρ_2 and the outlet velocity v_2. It is only problematic that iterations are required.

Finalised Nozzle (Stator) and Rotor Equations for Impulse Turbine

Given...

$$r_{blade} = r_{first-blade} * (1.05)^j$$

Impulse Turbine Angular Momentum Balance

$$I \frac{\partial \omega}{\partial t} = -\frac{(N^2 B^2 A^2)}{\eta R_{load}} \omega \sum_j \cos^2(\omega t + b_j)$$
$$+ \sum_j 1.05^j \dot{m} r_{first-blade} (v_{inlet,j} \cos \alpha_{inlet} - 1.05^j r_{first-blade} \omega)(1 + k_b)$$

Stator/Nozzle Balances

$$h_1 - h_2 = 0.5\left(\left(\frac{\rho_1 A_1}{\rho_2 A_2}\right)^2 - 1\right) v_1^2$$

$$\frac{P_1 A_1 - P_2 A_2}{\dot{m}} = \left(\frac{\rho_1 A_1}{\rho_2 A_2} - 1\right) v_1$$

Note that the nozzle area can vary independently from r_{blade}.

Isentropic Correlations and Algorithm

(h,s) algorithm The above constitute mass and energy balances along with momentum balances.

However, we need a way for the isentropic balances to be put in. Here is one algorithm:

First, we use

$$h_1 - h_{predicted} = 0.5\left(\left(\frac{\rho_1 A_1}{\rho_{2,old} A_2}\right)^2 - 1\right) v_1^2$$

Then we use the $h_{predicted}$ to guess the next thermodynamic state.

As the process is isentropic,

$$s_1 = s_2$$

The predicted state is:

$$\rho_{2,\text{guess}} = \rho(h_{predicted}, s_1)$$

Using this density, we can guess pressure P_2,

$$\frac{P_1 A_1 - P_2 A_2}{\dot{m}} = \left(\frac{\rho_1 A_1}{\rho_2 A_2} - 1\right) v_1$$

We can iterate until the ρ_2 reaches some steady state.

(p,s) algorithm Now, this uses the (h, s) backward equations in the steam table. This is good, except that (h, s) only covers part of the steam table. For the entire steam tables, (p, s) equations work better.

How would we do this?

First, we use

$$\frac{P_1 A_1 - P_{predicted} A_2}{\dot{m}} = \left(\frac{\rho_1 A_1}{\rho_{2,old} A_2} - 1\right) v_1$$

As the process is isentropic,

$$s_1 = s_2$$

$$\rho_{2,\text{guess}} = \rho(P_{predicted}, s_2) = \rho(P_{predicted}, s_1)$$

Using this density, we can guess h_2

$$h_1 - h_{predicted} = 0.5\left(\left(\frac{\rho_1 A_1}{\rho_{2,old} A_2}\right)^2 - 1\right) v_1^2$$

I tried using the above, didn't quite work. I'm changing the $h_{prediction}$ equation to make it more tightly coupled.

Now, we can use the new enthalpy, in another equation, so that we update the pressure we can actually split

$$h_1 - h_{predicted} = 0.5\left(\frac{\rho_1 A_1}{\rho_{2,old} A_2} - 1\right) v_1 \left(\frac{\rho_1 A_1}{\rho_{2,old} A_2} + 1\right) v_1$$

Substituting the pressure equation in:

$$h_1 - h_{predicted} = 0.5\frac{P_1 A_1 - P_{predicted} A_2}{\dot{m}}\left(\frac{\rho_1 A_1}{\rho_{2,old} A_2} + 1\right) v_1$$

Still didn't work, but actually what I forgot was to update the pressure again using the isentropically predicted pressure.

$$\frac{P_1 A_1 - P_{predicted} A_2}{\dot{m}} = \left(\frac{\rho_1 A_1}{\rho_{2,guess} A_2} - 1\right) v_1$$

Blade Velocity/Friction coefficient Also note:

$k_b = v_{\text{nozzle inlet next stage}}/v_{\text{nozzle outlet last stage}}$

Simplified P-S Algorithm for Stator - Rotor

Now, I note I have been experiencing bugs, in that previously, when (p,s) and (h,s) algorithms are used, the equations did not converge satisfactorily. The nozzle equations became such that entropy decreased rather than stayed the same. And instead of pressure dropping, it also increase in a most peculiar manner, much to my frustration.

One way is to opt for simpler equations. One where we neglect kinetic energy of inlet. In doing so, we leave the energy out of the iteration algorithm.

stator nozzle We noted before that:

$$\left(h + 0.5v^2\right)_1 = \left(h + 0.5v^2\right)_2$$

In some derivations, we simplified things by neglecting kinetic energy of the steam initially, relative to enthalpy drop as is done here:

$$h_1 = h_2 + 0.5v_2^2$$

$$h_{inlet} = h_{outlet} + 0.5v_{outlet}^2$$

Now, the momentum balance is:

$$\frac{P_1A_1 - P_2A_2}{\dot{m}} = \left(\frac{\rho_1A_1}{\rho_2A_2} - 1\right)v_1$$

Noting again:

$$\rho_1A_1v_1 = \rho_2A_2v_2$$

We can derive:

$$\frac{P_1A_1 - P_2A_2}{\dot{m}} = \left(\frac{\rho_1A_1}{\rho_2A_2} - 1\right)\frac{\dot{m}_1}{\rho_1A_1}$$

$$\frac{P_1A_1 - P_2A_2}{\dot{m}^2} = \left(\frac{1}{\rho_2A_2} - \frac{1}{\rho_1A_1}\right)$$

Or,

$$P_1A_1 - P_2A_2 = \dot{m}^2\left(\frac{1}{\rho_2A_2} - \frac{1}{\rho_1A_1}\right)$$

$$P_1A_1 + \frac{\dot{m}^2}{\rho_1A_1} = P_2A_2 + \frac{\dot{m}^2}{\rho_2A_2}$$

From this, the right hand side is unknown, the left hand side is known.
The algorithm can be as follows:

1. Find out thermodynamic state entropy s_1 using h_1, p_1
2. $s_2 = s_1$ for isentropic valve
3. guess p using a bisection algorithm with bounds between P_1 and 0 Pa
4. calculate ρ_2 and substitute
5. calculate residual
6. repeat until suitable P_2 is found
7. escape outside iteration loop
8. calculate v_2 using mass balance

9. calculate h_2 using energy balance

Now, with this algorithm, I tried a bisection root finding algorithm and was running into problems with root finding for the following equation:

$$P_1 A_1 + \frac{\dot{m}^2}{\rho_1 A_1} - P_2 A_2 + \frac{\dot{m}^2}{\rho_2 A_2} = 0$$

I was getting problems at step 3-6 as the bisection algorithm requires that the equation should yield a negative and positive value. However, for cases where there are no roots, or more than one root, or where the bounds for pressure are chosen poorly, the force balance equation can yield both positive or negative values.

If we define LHS and RHS to be the left hand side and right hand side of this force balance:

$$P_1 A_1 + \frac{\dot{m}^2}{\rho_1 A_1} = P_2 A_2 + \frac{\dot{m}^2}{\rho_2 A_2}$$

If we plot LHS - RHS for various inlet pressures using Rust, we get if plotting against the outlet pressure, figure 1.2:

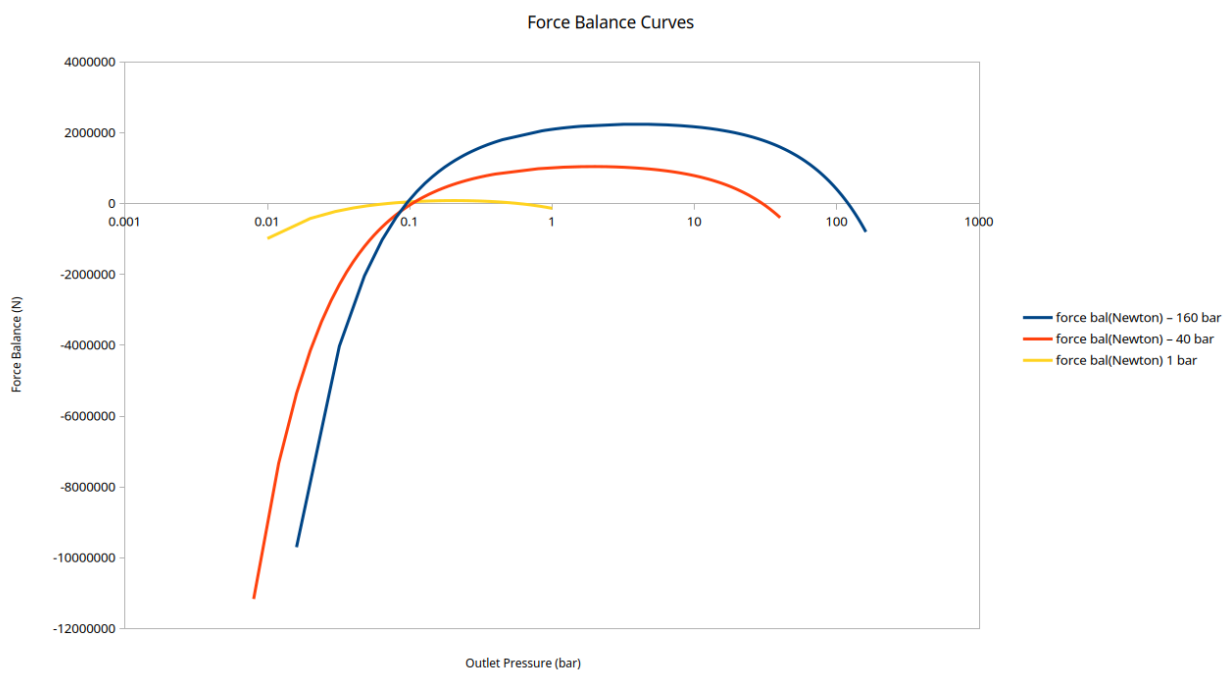

Figure 1.2: Force Balance Diagram

Noting the shape of this curve in figure 1.2, for various pressures, there are usually two roots. The shape of the curve is similar for all. Not quite a parabola, but like an upside down quartic curve.

For this, it looks like a Newton Raphson algorithm may work, at least for a first guess. Then perform some sort of crawling until proper bisection can be performed. So it's a hybrid of Newton Raphson and bisection.

Sonic Flow and Transonic Flow

Nozzle Balances for Mach < 0.7 and Mach > 1.3

Now, I tried all this, and I found that the mass, energy and momentum balances cannot really find a solution due to choked flow phenomenon, where compressible effects cannot be ignored. It turns out that this is very important especially during LOCA accidents. Hence, it is important to model for both steam/gas turbines and loss of coolant accidents (LOCA) in reactors.

In such flows, we have choked flows, where we can assume isentropic nozzles. Or we may have shockwaves, where entropy generation is present when doing supersonic/transonic flow. I use Cengel's 8th edition of thermodynamics to study this [Cengel and Boles, 2002]. This was done during the weekend of 28/29th march 2026.

In equation 17-16 and 17-17 of Cengel's thermodynamics textbook, we have:

$$\frac{dA}{A} = \frac{dP}{\rho v^2}(1 - \mathrm{Ma}^2) \tag{1.6}$$

$$\frac{dA}{A} = -\frac{dv}{v}(1 - \mathrm{Ma}^2) \tag{1.7}$$

Why does mach number come in?

Mach number, comes in due to the importance of speed of sound. The speed of sound is characterised by **isentropic** pressure waves.

From the Maxwell relations:

$$Tds = dh - VdP$$

$$Tds = dh - \frac{dP}{\rho}$$

Under isentropic assumption, $ds = 0$

$$dh = \frac{dP}{\rho}$$

From the derivation of speed of sound, considering energy balance on the front of a sound wave:

$$dh = cdV$$

And the mass balance:

$$cd\rho = \rho dV$$

We eliminate h and dV from the equation under isentropic conditions:

$$dh = \frac{dP}{\rho} = cdV$$

And we substitute dV

$$\frac{dP}{\rho} = cdV = c^2\frac{d\rho}{\rho}$$

So we get

$$\frac{dP}{\rho} = c^2\frac{d\rho}{\rho}$$

Under ds = 0:

$$\frac{dP}{d\rho} = c^2$$

$$c^2 = \left(\frac{dP}{d\rho}\right)_s$$

Now, this will be important for flow in nozzles.
We start with mass balance:

$$d(\dot{m}) = 0$$

$$d(\rho A v) = 0$$

$$Avd\rho + v\rho dA + \rho Adv = 0$$

divide by $\dot{m} = \rho Av$:
This is general mass balance equation:

$$\frac{d\rho}{\rho} + \frac{dA}{A} + \frac{dv}{v} = 0$$

Then let's do energy balance:

$$d(h + v^2/2) = 0$$

$$dh + vdv = 0$$

Now, we use maxwell relation again for isentropic case:

$$Tds = dh - \frac{dP}{\rho}$$

Under isentropic assumption, $ds = 0$

$$dh = \frac{dP}{\rho}$$

$$dh + vdv = \frac{dP}{\rho} + vdv = 0$$

Thus we get differential equations for flow nozzles, not going to derive here...:

$$\frac{dA}{A} = \frac{dP}{\rho v^2}(1 - \mathrm{Ma}^2) \tag{1.8}$$

$$\frac{dA}{A} = -\frac{dv}{v}(1 - \mathrm{Ma}^2) \tag{1.9}$$

Implementing this in Rust was quite trivial, as we can rearrange the equations to get dv/v and dA/A.

The problem now becomes when dealing with the term $1 - \mathrm{Ma}^2$ near transonic speeds. Where this number approaches singularity. (ie 0)

Of course, dA/dv and dA/dp approach zero. Which is okay. But my goal is to find velocity change and pressure change. I am interested in dp and dv rather than dA, which makes this rather problematic.

In these regimes, one can approximate nozzle flow as isentropic.

Nozzle Balances for 0.7 < Mach < 1.3, Transonic regime

Now, for these kinds of problems, I'm overall trying to find **pressure and velocity** at throat, given area for a transonic flow situation in Mach number from 0.7 to 1.3.

If coming from a subsonic velocity, for converging nozzles, flow choking will occur where the maximum velocity at the throat is Mach 1. In such a case, velocity at the throat is going to be the speed of sound.

To accelerate the flow further, one must make the nozzle diverge after the throat. This is done with a diverging nozzle for supersonic flow.

Nozzle tthrough to Throat Subsonic case, area decreasing In this case, we will need a function to determine acceleration up to this (critical point). Combining energy and mass balance:

$$h_1 + \frac{v_1^2}{2} = h_2 + \frac{v_2^2}{2}$$

The mass balance is:

$$\rho_1 A_1 v_1 = \rho_2 A_2 v_2$$

Combining momentum and mass balance:

$$P_1 A_1 + \frac{\dot{m}^2}{\rho_1 A_1} = P_2 A_2 + \frac{\dot{m}^2}{\rho_2 A_2}$$

Now, this momentum equation is valid for short nozzles, where force by the nozzle surfaces on the fluid is neglibible. But in general, that is not the case. Most of the time in Cengel's textbook, we don't use force balances for that reason, except when considering shockwaves where the force balance is used for the Rayleigh line (momentum and mass balance).

These are the governing equations. Now, with all these variables, we are trying to check for a choked flow condition. Under choked flow, decreasing outlet pressure beyond a certain number will not increase velocity. This is the critical pressure.

In such choked flow, the inlet and outlet stagnation enthalpies will match. The velocity at the outlet enthalpy will be speed of sound:

$$h_1 + \frac{v_1^2}{2} = h_2 + \frac{c^2}{2}$$

Given this, and the inlet pressure, we should be able to have $s_1 = s_1(h_1, p_1)$ from (p,h) equations. And from isentropic relations, $s_1 = s_2$, so we can obtain state 2 from (h, s) relations. This is for compressible flow only.

Note:, (h,s) equations were not quite tested for tampines steam tables, as of v0.1.5, but now in v0.1.6 onwards, these are tested. And for low quality steam, below the threshold entropy of about 5.2 kJ/(kg K), this defaults to iterative (p,h) algorithm with bisection.

So with (h,s) equations, we obtain the outlet enthalpy. With outlet enthalpy and entropy, we will be able to get outlet pressure, and density P_2 and ρ_2 at choked flow. This is done using critical pressure ratio functions, which determine the thermodynamic state at the throat.

Problem statement and solution sequence Now, the problem statement for a nozzle (stator) plus rotor section is this.

- We have two control volumes between nozzle/rotor pair
- CV-1 before nozzle has known (p,h) state, and stagnation state
- CV-2 after the rotor, has known (p,h) state
- steam goes from CV-1 to throat, accelerates
- steam accelerates further or goes through shock before diverging nozzle exit
- steam goes under no pressure decrease in impulse turbine, only velocity decrease

- steam isentropically returns to stagnation state before entering CV-2
- mass flowrate is determined iteratively after thermodynamics

For these, at least two areas must be known. First is the throat area, second is the nozzle exit area.

We can first solve for whether choked flow happens in the throat area. We can neglect the kinetic energy. We do so using this code in converging_diverging_nozzle.rs:

```
        /// This gives the mass flowrate for choked flow given the area of a throat
/// and the inlet stagnation properties (negligble KE)
#[inline]
pub fn get_choked_flow_massrate_and_state_from_stagnation_properties_and_area (
    p0: Pressure,
    h0: AvailableEnergy,
    a_throat: Area,
) -> (MassRate, TampinesSteamTableCV) {

    let ref_vol = Volume::new::<cubic_meter>(1.0);
    // now we have inlet stagnation state
    let state_0 = TampinesSteamTableCV::new_from_ph(p0, h0, ref_vol);


    // calculate entropy, because process should be isentropic,
    // entropy is constant
    let s0 = state_0.get_specific_entropy();


    // calculate critical pressure
    let critical_pressure_ratio: Ratio =
        state_0.get_critical_pressure_ratio();

    // this is critical pressure for mach 1
    let p_critical = critical_pressure_ratio * p0;
    let s_throat = s0;

    let state_choked = TampinesSteamTableCV::new_from_ps(
        p_critical, s_throat, ref_vol
    );

```

```
    let c = state_choked.get_speed_of_sound();

    let rho_choked = state_choked.get_rho();

    let massrate: MassRate = c * rho_choked * a_throat;


    return (massrate, state_choked);
}
```

Now, this mass flowrate will be the upper bound mass flowrate for choked flow, the lower bound, is of course, 0 kg/s. This is the nozzle calculation.

Now, for the diverging part, we will also solve similar equations. But the flow may or may not be isentropic due to presence of shocks. This will depend on the outlet pressure. The outlet pressure is indeed fixed, but the throat pressure is not.

However, given a throat velocity, we will have a fixed throat pressure and mass flowrate.

We need to break down into a few cases what may happen.

Case 0: Subsonic isentropic flow throughout In this case, we don't have choked flow. The flow simply decelerates towards the exit with appropriate exit pressure. Flow is subsonic, isentropic. No shocks or anything interesting.

In such a case, the exit velocity is limited by the exit area rather than anything.

With stagnation properties p0, h0, and exit area,

$$h_0 = h_{exit} - 0.5 * v_{exit}^2$$
$$s_0 = s_{exit}$$

$$\dot{m} = \rho_{exit} a_{exit} v_{exit}$$

Solving these equations simultaneously will allow for mass flowrate to be determined.

In the subsonic flow case, entropy is fixed, specifying v_{exit} will determine h_{exit}, and ρ_{exit} with the use of the isentropic relations. Hence, one more constraint is required.

The most straightforward way in this case is to use a (p,s) flash, if we indeed confirm that the flow is isentropic. With these, h_{exit} is found, then v_{exit} will be found via the energy balance.

ρ_{exit} will be determined by this exit state, and then from therein, we get the mass flowrate.

So the steps are:

- Determine that flow is subsonic (no choked flow)
- Find end-state at exit using (p,s) flash
- Using end state, find v_{exit} using $h_0 = h_{exit} + 0.5 v_{exit}^2$

- Using v_{exit} and ρ_{exit} from end state, find mass flowrate

With these steps in mind, it should be pretty trivial

Case 1: Choked, and Pressure at throat is less than exit pressure In this case, it is quite straightforward, there will be decelartion towards subsonic flow. This can happen without shocks forming. Now, if the pressure drops below some threshold value, there will be normal shocks forming in the system, and it is no longer isentropic flow.

TBD

Case 2: Choked and nozzle results in Supersonic flow In this case, it is also relatively straightforward, the exit pressure is low enough that flow can expand and acceleaate through the nozzle without any shocks forming.

How low is low enough?

stability issues and local time stepping (Courant Number)

Reaction Turbines

Reaction Turbines lose pressure both within the stator and the rotor. [To be continued...]

Now we have the stator and rotor equations, we need to consider how they will connect to the control volumes

Simplifications

Note, for loss coefficients, they are in Appendix A1 to A6 of Agromayor's equations [Agromayor and Nord, 2019]. We are ignoring turbine losses in this work.

1.4 Converting forms to Mass Flowrate

It is often more convenient to convert the momentum equations to be in terms of mass flowrates. This is for 1D case.

To do so, we transform this by consider this 3D equation.

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.10}$$

We need to consider just the x component, for a simple 1D case.

Then multiply by the cross sectional area.

1.5 Approach to Solving nonlinear Equations

Jasak in his PhD thesis outlines the methodology for solving this set of equation [Jasak, 1996] on page 143-149. These were the foundations for what would eventually become OpenFOAM Jasak, Jemcov, Tukovic, et al., 2007 licensed under GPLv3.

The momentum equation is quadratic in u, which makes the equation solution nonlinear. Solving this nonlinear equation would either need a nonlinear solver such as Newton Raphson method, or solving a linearised form of these equations, and iterating to convergence. The latter was chosen [Jasak, 1996].

$$\nabla \bullet (\mathbf{uu}) = \sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f = \sum_f F \mathbf{u}_f = a_p \mathbf{u}_p + \sum_N a_N \mathbf{u}_n \tag{1.11}$$

Finite Volume Discretisaion, Face and Centroid Properties

Section 3.3.1 within Jasak's thesis explains these terms.

First, we need to understand that Gauss's theorem is used in Finite Volume analysis, which is the bedrock of OpenFOAM solvers. $\sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f$ represent the surface vector $\mathbf{S}$ and this will have a dot product with the velocity at the faces of the control volume $\mathbf{u}_f$.

Note that the subscript f represents values at the faces.

$\sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f$ can be written in shorthand $\sum_f F \mathbf{u}_f$.

Now, we may not have the values of the velocities or fluxes at the faces. So we have to estimate it based on the properties at the centroid of the cell denoted p, and those of the neighbours denoted n.

Thus, the surface values denoted f are estimated based on the values at the cell centroids p and n based on some kind of interpolation scheme. This can be linear interpolation, or some upwind differencing scheme. This is described in page 81 of Jasak's thesis [Jasak, 1996].

With linear interpolation at the face, the solution can become unbounded. With upwind differencing, the solution is bounded, but diffusive. Much of the discussion revolves around how to estimate properties at the faces. This is why a whole host of differencing schemes comes up. Eg. at page 110 of Jasak's thesis [Jasak, 1996].

Linearisation and Predictor values

Given a very nonlinear system of equations. It was desirable to consider solving a linearised version of the momentum equations. We want to solve for the velocity $\mathbf{u}^{t+\Delta t}$ at the next timestep. But we only know $\mathbf{u}^t$.

We know that the face fluxes should be based on values in the cell centroids for both the cell p and that of the neighbouring cell n, hence we use:

$$a_p \mathbf{u}_p + \sum_N a_N \mathbf{u}_n$$

However, we have $\mathbf{uu}$ terms. To solve this problem and linearise this, we can use $\mathbf{u}^t\mathbf{u}^{t+\Delta t}$. Basically, $\mathbf{u}^t$ is known. This is the velocity at the previous timestep, and is therefore a constant. This will then help form the respective a_p and a_n constants so that the equations are linear.

Let's go back to our equations:

$$\frac{\partial(\rho\mathbf{u})}{\partial t} + \nabla\cdot(\rho\mathbf{uu}) = -\nabla P + \nabla\cdot\tau + \rho g \tag{1.12}$$

In the incompressible assumption,

$$\frac{\partial(\mathbf{u})}{\partial t} + \nabla\cdot(\mathbf{uu}) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \tag{1.13}$$

We can do time discretisation:

$$\frac{\mathbf{u}^{t+\Delta t} - \mathbf{u}^t}{\Delta t} + \nabla\cdot(\mathbf{uu}) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \tag{1.14}$$

Now, remember, based on divergence theorem we need fluxes or properties at the faces:

$$\frac{\mathbf{u}^{t+\Delta t} - \mathbf{u}^t}{\Delta t} + \sum_f \mathbf{S}\cdot\mathbf{u}_f\mathbf{u}_f = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \tag{1.15}$$

And since we cannot find properties at the faces, we have to estimate them using properties at the centroids.

Just to be clear, let's label the velocities at their proper locations, and then linearise the equation using some semi implicit time discretisation:

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S}\cdot\mathbf{u}_f^t\mathbf{u}_f^{t+\Delta t} = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \tag{1.16}$$

The fluxes at the faces can be further estimated using some interpolation scheme. The easiest to understand is a linear scheme. Herein, we use some weighting factor to estimate the flux at the face between the cell and the other cell.

$$\mathbf{u}_f^t = w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t$$

Substituting this back:

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S}\cdot(w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t)(w\mathbf{u}_p^{t+\Delta t} + (1-w)\mathbf{u}_n^{t+\Delta t}) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \tag{1.17}$$

To make life easier for us, we can choose not to even bother with explicitly solving for the velocities of the neigbours

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S} \cdot (w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t)(w\mathbf{u}_p^{t+\Delta t} + (1-w)\mathbf{u}_n^t) = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \quad (1.18)$$

Messy as the equation is, we now have a linear equation in velocities for each centroid at the next timestep $u_p^{t+\Delta t}$ and $u_n^{t+\Delta t}$.

We can then simplify this equation as such:

$$a_p \mathbf{u}_p^{t+\Delta t} = \mathbf{H}(\mathbf{u}^t) - \nabla P/\rho + \nabla \cdot \tau/\rho + g \quad (1.19)$$

$$\mathbf{u}_p^{t+\Delta t} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \quad (1.20)$$

Of course, the estimates for $\mathbf{u}^{t+\Delta t}$ are not going to be accurate. They are initial guess or **predicted** values.

We need to do some further correction to these predicted values in order to arrive at the correct answer.

Corrector steps

So far, we've only bothered with the momentum equation and ignored mass balance (or continuity in this case). Thus, there is no guarantee that the mass balance is fulfilled.

The mass balance is:

$$\nabla \cdot \mathbf{u}_p = 0$$

We then apply Gauss's divergence theorem to obtain:

$$\nabla \cdot \mathbf{u}_p = \sum_f \mathbf{S} \cdot \mathbf{u}_f = 0$$

Now again, how do we deal with the face fluxes again? We need to interpolate.

Suppose again that we can use some weighting factor (linear interpolation scheme), that on a face between a point and its neighbouring cell:

$$u_f = wu_p + (1-w)u_n$$

$$\mathbf{u}_p^{t+\Delta t} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p$$

since

$$\mathbf{u}_p^{t+\Delta t} = 0$$

Then:

$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \right) = 0$$

Moving the pressure term to the right:

$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \right) = \nabla \cdot \frac{P/\rho}{a_p}$$

This is equation 3.141 in page 146 of Jasak's thesis [Jasak, 1996]. Of course, τ and g were not included.

Now, if we apply divergence theorem on the H terms:

$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} \right) = \sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} \right)_f$$

We also need to discretise the pressure side

$$\nabla \cdot \frac{P/\rho}{a_p} = \sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p} \right)_f$$

With this discretisation, we obtain

$$\sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} \right)_f = \sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p} \right)_f$$

With the interpolation:

$$u_f = w u_p + (1 - w) u_n$$

the matrices can be set up. This is equation 3.143 in Jasak's thesis [Jasak, 1996].

So basically, the continuity equation is combined with the momentum equation to give a combined corrector equation.

1.6 PISO Algorithm

Pressure Implicit Splitting of Operators (PISO).

This includes a few steps, outlined in Jasak's thesis [Jasak, 1996]:

Step 1

This is the momentum predictor step, we can predict a velocity field using the previous steps

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \tag{1.21}$$

Step 2

With this, we can formulate a new $\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}$:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} \tag{1.22}$$

With this, we can substitute into the mass balance (pressure equation):

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f = \sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}\right)_f$$

This allows for a new pressure field. Note that the gravity terms are missing because hydrostatic pressure is ignored.

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

It is conceivable that this results in some small matrix where pressure fields are simultaneously solved.

This is the pressure solution within Jasak's thesis page 147 [Jasak, 1996].

Step 3

We now have a new pressure field based on mass balance:

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

And a new predicted H field based on the initial predictor step:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

Now, the predictor H field is not guaranteed to allow continuity with the old velocity, but since it was used to generate a new pressure field using the continuity equation, it should be able to guarantee creation of a new velocity field which will satisfy continuity equation.

$$\mathbf{u}_p^{corrected} = \frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} - \left(\frac{\nabla P/\rho}{a_p}\right)_{\text{corrected pressure field}} \tag{1.23}$$

repeated iterations

This forms the basis for iterating through both mass and momentum balances. This is one round of PISO prediction-correction.

We may iterate this several times until the residual is small between consecutive iterations.

Now, it must be noted that during PISO, we do NOT repeat step 1 to recalculate as mentioned in page 148 Jasak's thesis Jasak, 1996:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

The H term is kept constant through the whole correction sequence and changed in only the next momentum predictor [Jasak, 1996].

1.7 SIMPLE Algorithm

The SIMPLE algorithm is similar, except that it is solving for steady state solutions, and uses large time step.

The key difference is rather than skip Step 1 within loops, we repeat step 1 for momentum predictor. This is the key difference between PISO and SIMPLE. It is that we run the momentum predictor step every time in SIMPLE. In this regard, it is more important to re-calculate H in every iteration so that non-linearities are accounted for.

Hence, between iterations, there is no need to fully resolve pressure-velocity coupling. Which means between iterations, one does not need to actively ensure that they are solved simultaneously.

Step 1

Same as PISO, this is the momentum predictor step, we can predict a velocity field using the previous steps

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \tag{1.24}$$

However, before going to step 2, we

Step 2

Also same as PISO, we do the pressure equation step

With this, we can formulate a new $\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}$:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} \tag{1.25}$$

With this, we can substitute into the mass balance (pressure equation):

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f = \sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}\right)_f$$

This allows for a new pressure field. Note that the gravity terms are missing because hydrostatic pressure is ignored.

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

It is conceivable that this results in some small matrix where pressure fields are simultaneously solved.

This is the pressure solution within Jasak's thesis page 147 [Jasak, 1996].

Step 3: Momentum Corrector

We now have a new pressure field based on mass balance:

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

And a new predicted H field based on the initial predictor step:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

Now, the predictor H field is not guaranteed to allow continuity with the old velocity, but since it was used to generate a new pressure field using the continuity equation, it should be able to guarantee creation of a new velocity field which will satisfy continuity equation.

$$\mathbf{u}_p^{corrected} = \frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} - \left(\frac{\nabla P/\rho}{a_p}\right)_{\text{corrected pressure field}} \tag{1.26}$$

repeated iterations

This forms the basis for iterating through both mass and momentum balances. This is one round of SIMPLE prediction-correction.

We may iterate this several times until the residual is small between consecutive iterations.

Now, it must be noted that previously during PISO, we do NOT repeat step 1 to recalculate as mentioned in page 148 Jasak's thesis Jasak, 1996:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

Now, in SIMPLE, we do change H during subsequent iterations in the momentum predictor step [Jasak, 1996].

Dr Aiden's YouTube videos were helpful in understanding this regard too.

Relaxation Factors

However, between each iteration, timesteps may be large. Hence, the solution may experience some form of instability. Therefore, there is the introduction of relaxation factors which updates the velocity slowly between successive momentum predictor steps.

Jasak mentions the use of velocity under-relaxation factors α_u. as well as pressure under-relaxation factors α_p.

He outlined the following steps on page 148-149 of his thesis [Jasak, 1996]. For pressure:

$$p^{new} = p^{old} + \alpha_p(p^{predictor} - p^{old})$$

For momentum, a similar thing applies. Though for momentum α_u is usually around 0.8 and for pressure α_p is around 0.2. Jasak writes this in page 149 of his thesis [Jasak, 1996].

1.8 PIMPLE Algorithm

For the PIMPLE algorithm, this is a merger between PISO and SIMPLE algorithms [Greenshields and Weller, 2022].

Basically, every SIMPLE iteration now has the option to perform a set number of PISO iterations as well. That is perform the corrector steps multiple times per SIMPLE iteration. The PISO loops are known as inner loops, whereas the outer loops are known as SIMPLE loops.

It is then up to the user to know how many loops are required.

With this understanding, it is good to try and apply this thought process to our four control volume situation.

1.9 Application to Rankine Cycle

Mass, Energy and Momentum Balance Equations

Consider four control volumes. We consider the following PDEs:

$$\frac{\partial m}{\partial t} = \sum_i m_i \tag{1.27}$$

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.28}$$

$\nabla \cdot \tau$ represents shear, ρg represents the gravity body force, $-\nabla P$ represents the pressure gradient.

$$V_j \frac{\partial \rho_i h_i}{\partial t} + \sum_j \dot{m}_j h_j = Q_i - W_i + \sum_j \tau_j \cdot \mathbf{u}_j \tag{1.29}$$

For a 1D case, we can consider Equations from Modelica. This was the excerpt from my dissertation [Ong, 2024]:

The mass balance equation for a 1D geometry is [Casella, Leva, et al., 2003].

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}}{\partial x} = 0 \tag{1.30}$$

Where A_{XS} is the cross sectional area, ρ is the density of the differential volume dV, $\dot{m}$ is the mass flow rate, and x is the length coordinate in the direction of flow.

The momentum balance is [Casella, Leva, et al., 2003]:

$$\frac{\partial \dot{m}}{\partial t} + \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}|\dot{m}| + \rho g A_{XS} \frac{\partial z}{\partial x} + A_{XS} \frac{\partial P}{\partial x} = 0 \tag{1.31}$$

In Equation 1.31, in addition to the terms already defined for Equation 1.30, we have Fanning friction factor $f_{fanning}$, wetted perimeter P_w, acceleration due to gravity g, height for calculating hydrostatic pressure z and pressure P. The absolute mass flow rate $\dot{m}$ is used to account for flow reversal in the momentum equation. Additionally the derivative $\frac{\partial \dot{m}}{\partial t}$ is often set to zero to avoid fast pressure oscillations [Casella, Leva, et al., 2003]. The timescale of these oscillations is quite often connected to the speed of sound in that medium.

The energy balance is [Casella, Leva, et al., 2003]:

$$\rho A_{XS} \frac{\partial h_{enthalpy}}{\partial t} + A_{XS} \frac{\partial P}{\partial t} = -\rho A_{XS} u \frac{\partial h_{enthalpy}}{\partial x} + q_{wall} \frac{A_{wall}}{L} \tag{1.32}$$

In Equation 1.32, in addition to the previously defined terms for Equation 1.30 and 1.31, I define $h_{enthalpy}$ for specific enthalpy per unit mass for the control volume, average fluid velocity u and q_{wall}, the heat flux received or lost through lateral surfaces. $h_{enthalpy}$ is denoted as such to prevent confusion with heat transfer coefficient, also commonly denoted h. $\frac{A_{wall}}{L}$ is the heated surface area of the wall A_{wall} per unit length, where L is the length of the 1D geometry. For tubes, $\frac{A_{wall}}{L} = P_w$ [Casella, Leva, et al., 2003].

If we consider the 1D balances:

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}}{\partial x} = 0$$

$$\frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}|\dot{m}| + \rho g A_{XS} \frac{\partial z}{\partial x} + A_{XS} \frac{\partial P}{\partial x} = 0$$

$$\rho A_{XS} \frac{\partial h_{enthalpy}}{\partial t} + A_{XS} \frac{\partial P}{\partial t} = -\rho A_{XS} u \frac{\partial h_{enthalpy}}{\partial x} + q_{wall} \frac{A_{wall}}{L}$$

Of course, for the equation of state, we can use a real gas equation or steam table. That is done in tampines-steam-tables.

Chapter 2

rhoPimpleFoam in OpenFOAM

For the algorithm considered, we are going to translate from rhoPimpleFoam to what we want:

```
/*
    ------------------------------------------------------------------------*\
  ========= |
  \\ / F ield | OpenFOAM: The Open Source CFD Toolbox
   \\ / O peration |
    \\ / A nd | www.openfoam.com
     \\/ M anipulation |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2019 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
    for more details.
```

```

    You should have received a copy of the GNU General Public License
    along with OpenFOAM. If not, see <http://www.gnu.org/licenses/>.

Application
    rhoPimpleFoam

Group
    grpCompressibleSolvers

Description
    Transient solver for turbulent flow of compressible fluids for HVAC and
    similar applications, with optional mesh motion and mesh topology changes.

    Uses the flexible PIMPLE (PISO-SIMPLE) solution for time-resolved and
    pseudo-transient simulations.

Note
   The motion frequency of this solver can be influenced by the presence
   of "updateControl" and "updateInterval" in the dynamicMeshDict.

\*---------------------------------------------------------------------------*/

#include "fvCFD.H"
#include "dynamicFvMesh.H"
#include "fluidThermo.H"
#include "turbulentFluidThermoModel.H"
#include "bound.H"
#include "pimpleControl.H"
#include "pressureControl.H"
#include "CorrectPhi.H"
#include "fvOptions.H"
#include "localEulerDdtScheme.H"
#include "fvcSmooth.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

int main(int argc, char *argv[])
{
```

```
    argList::addNote
    (
        "Transient solver for compressible turbulent flow.\n"
        "With optional mesh motion and mesh topology changes."
    );

    #include "postProcess.H"

    #include "addCheckCaseOptions.H"
    #include "setRootCaseLists.H"
    #include "createTime.H"
    #include "createDynamicFvMesh.H"
    #include "createDyMControls.H"
    #include "initContinuityErrs.H"
    #include "createFields.H"
    #include "createFieldRefs.H"
    #include "createRhoUfIfPresent.H"

    turbulence->validate();

    if (!LTS)
    {
        #include "compressibleCourantNo.H"
        #include "setInitialDeltaT.H"
    }

    // * * *
```

```
            (
                new volScalarField
                (
                    "divrhoU",
                    fvc::div(fvc::absolute(phi, rho, U))
                )
            );
        }

        if (LTS)
        {
            #include "setRDeltaT.H"
        }
        else
        {
            #include "compressibleCourantNo.H"
            #include "setDeltaT.H"
        }

        ++runTime;

        Info<< "Time = " << runTime.timeName() << nl << endl;

        // --- Pressure-velocity PIMPLE corrector loop
        while (pimple.loop())
        {
            if (pimple.firstIter() || moveMeshOuterCorrectors)
            {
                // Store momentum to set rhoUf for introduced faces.
                autoPtr<volVectorField> rhoU;
                if (rhoUf.valid())
                {
                    rhoU.reset(new volVectorField("rhoU", rho*U));
                }

                // Do any mesh changes
                mesh.controlledUpdate();

                if (mesh.changing())
                {
                    MRF.update();

```

```
                if (correctPhi)
                {
                    // Calculate absolute flux
                    // from the mapped surface velocity
                    phi = mesh.Sf() & rhoUf();

                    #include "correctPhi.H"

                    // Make the fluxes relative to the mesh-motion
                    fvc::makeRelative(phi, rho, U);
                }

                if (checkMeshCourantNo)
                {
                    #include "meshCourantNo.H"
                }
            }
        }

        if (pimple.firstIter() && !pimple.SIMPLErho())
        {
            #include "rhoEqn.H"
        }

        #include "UEqn.H"
        #include "EEqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            if (pimple.consistent())
            {
                #include "pcEqn.H"
            }
            else
            {
                #include "pEqn.H"
            }
        }

        if (pimple.turbCorr())
        {
```

```
                turbulence->correct();
            }
        }

        rho = thermo.rho();

        runTime.write();

        runTime.printExecutionTime(Info);
    }

    Info<< "End\n" << endl;

    return 0;
}


// ************************************************************************
    ↪  //
```

Listing 2.1: rhoPimpleFoam.C

Discretisation

Considering the 1D control volume, and a 1D version of the divergence theorem, we can write:

$$\Delta x A_{XS} \frac{\partial \rho}{\partial t} + \Delta x \frac{\partial \dot{m}}{\partial x} = 0$$

$$V \frac{\partial \rho}{\partial t} + \sum_j \dot{m}_{j,face} = 0$$

As with before, we must estimate $\dot{m}_{j,face}$ using the centroid values. We can use upwinding interpolation, which would make the most sense.

That means, using p to denote the value at the centroid of the cell, and n to denote value at centroid of neighbouring cell.

At the face that is going to some adjacent cell,

If $\dot{m}_p \cdot \hat{S} > 0$:

$$\dot{m}_{j,face} = \dot{m}_p$$

else

$$\dot{m}_{j,face} = \dot{m}_n$$

$\hat{S}$ is the unit area surface normal vector that points away from the centroid of the cell.

Step 1: Density Equation

For temporal time stepping:

$$V\frac{\rho^{t+\Delta t} - \rho^t}{\Delta t} + \sum_j \dot{m}_{j,face} = 0$$

We have an explicit equation that allows us to determine a guess in density for the next iteration if we were to do it explicitly.

This is the first step in the rhoPimpleFoam algorithm. (see line 169)

If there are any flows in or out of here, we can incorporate it as well into this equation.

Of course, $\rho^{t+\Delta t}$ is not exactly the correct term, as we haven't completely solved and iterated the new density fields. $\rho^{predicted}$ should be the correct term. So what we have now is an intermediate value:

$$V\frac{\rho^{predicted} - \rho^t}{\Delta t} + \sum_j \dot{m}_{j,face} = 0$$

Step 2: Momentum Predictor Step

The momentum predictor step in OpenFOAM is as follows:

```

// Solve the Momentum equation

MRF.correctBoundaryVelocity(U);

tmp<fvVectorMatrix> tUEqn
(
        fvm::ddt(rho, U) + fvm::div(phi, U)
        + MRF.DDt(rho, U)
        + turbulence->divDevRhoReff(U)
        ==
        fvOptions(rho, U)
);
fvVectorMatrix& UEqn = tUEqn.ref();

UEqn.relax();
```

```

fvOptions.constrain(UEqn);

if (pimple.momentumPredictor())
{
        solve(UEqn == -fvc::grad(p));

        fvOptions.correct(U);
        K = 0.5*magSqr(U);
}
```

For the momentum predictor, we start with:

$$\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

Now, we know that the densities have been predicted based on mass balance, we can do a momentum predictor step. Note that we ignore the $\frac{\partial \dot{m}}{\partial t}$ term just like modelica. This makes it easier for us. The only thing for us to discrete here is the pressure equation. We will use divergence theorem again by integrating each term by volume.

However, the 1D form has already been integrated across the areas, so we only need do is over dx:

$$\Delta x\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\int dx\frac{\partial z}{\partial x} + A_{XS}\int dx\frac{\partial P}{\partial x} = 0$$

$$\Delta x\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\Delta z + A_{XS}\Delta P_{x,face} = 0$$

For this, we can do some pressure interpolation using some weighting factor α:

$$\Delta P_{x,face} = \alpha P_p + (1-\alpha)P_n$$

OpenFOAM tends to correlate α based on the distance from the cell centroid to the boundary. In the 1D case, α is simply based on the ratios of the lengths of each 1D cell.

For this, we can see

$$1-\alpha = \frac{\text{cell P length}}{\text{cell P length} + \text{cell N length}}$$

Hence,

$$\alpha = \frac{\text{cell N length}}{\text{cell P length} + \text{cell N length}}$$

Now, the density would be based on the previous density predictor step, but the pressures and other properties would be based on the previous timestep.

$$\Delta x \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}^{predicted} |\dot{m}^{predicted}| + \rho g A_{XS} \Delta z + A_{XS} \Delta P_{x,face} = 0$$

As for whether we should take the positive or negative answer of $|\dot{m}^{predicted}|$, that should depend on the sign of the total pressure change.

Now, it is evident that this is a nonlinear equation. So we want to linearise this equation just like for PIMPLE. We shall use the mass flowrates from the previous timesteps for the fanning friction factor and $\dot{m}$.

$$\Delta x \frac{f^t_{fanning} P_w}{2\rho^{predicted} A_{XS}^2} \dot{m}^t |\dot{m}^{predicted}| + \rho^{predicted} g A_{XS} \Delta z + A_{XS} \Delta P^t_{x,face} = 0$$

Now, solving for this is kind of cumbersome due to the presence of the modulus sign, it may be better to do:

$$\Delta x \frac{f^t_{fanning} P_w}{2\rho^{predicted} A_{XS}^2} \dot{m}^{predicted} |\dot{m}^t| + \rho^{predicted} g A_{XS} \Delta z + A_{XS} \Delta P^t_{x,face} = 0$$

Of course, if area and such were changing, we could use this from the last timestep. Nevertheless, this is a control volume, and we should not expect area to change.

This makes it relatively easy to obtain $\dot{m}^{predicted}$ as the steps are quite explicit.

Step 3: Energy Equation

With the predicted mass flowrates, and densities, we can then consider the energy equation.

$$\rho A_{XS} \frac{\partial h_{enthalpy}}{\partial t} + A_{XS} \frac{\partial P}{\partial t} = -\rho A_{XS} u \frac{\partial h_{enthalpy}}{\partial x} + q_{wall} \frac{A_{wall}}{L}$$

$$\rho A_{XS} \frac{\partial h_{enthalpy}}{\partial t} + A_{XS} \frac{\partial P}{\partial t} = -\dot{m} \frac{\partial h_{enthalpy}}{\partial x} + q_{wall} \frac{A_{wall}}{L}$$

Let us integrate across dx:

$$\rho V \frac{\partial h_{enthalpy}}{\partial t} + V \frac{\partial P}{\partial t} = -\sum_j \dot{m}_j h_j + q_{wall} A_{wall}$$

Now, this is tricky because we have two time derivatives. One based on pressure, one based on enthalpy. What do we do with this?

Based on the OpenFOAM EEqn.H in rhoPimpleFoam:

```
{
    volScalarField& he = thermo.he();

    fvScalarMatrix EEqn
    (
```

```
        fvm::ddt(rho, he) + fvm::div(phi, he)
      + fvc::ddt(rho, K) + fvc::div(phi, K)
      + (
            he.name() == "e"
          ? fvc::div
            (
                fvc::absolute(phi/fvc::interpolate(rho), U),
                p,
                "div(phiv,p)"
            )
          : -dpdt
        )
      - fvm::laplacian(turbulence->alphaEff(), he)
     ==
        fvOptions(rho, he)
    );

    if (MRF.active())
    {
        EEqn += fvc::div(MRF.phi(), p);
    }

    EEqn.relax();

    fvOptions.constrain(EEqn);

    EEqn.solve();

    fvOptions.correct(he);

    thermo.correct();
}
```

It appears we have to calculate a pressure gradient first. Not sure where we get that from though!

Now, for the **first** iteration, dpdt is zero, because there is no prior pressure from which to calculate it.

In subsequent steps though, p will be updated. And one can have a dpdt as

$$\frac{\partial p}{\partial t} = \frac{p^t - p^{t-\Delta t}}{\Delta t}$$

Equation of State:

Now, with this, we will then be able to solve for enthalpy at the next timestep.

Now, we have predicted a new density ρ, a new set of mass flowrates $\dot{m}$ and a new enthalpy h. With density and enthalpy, in theory, one should be able to obtain a new pressure field.

$$p = p(\rho, h)$$

For a ideal gas equation of state, this is quite easy if the heat capacity equation is straightforward, as pressure is explicitly in terms of specific volume $1/\rho$ and temperature.

However, for cubic equation of state, then we need to find temperature iteratively from h.

And for steam tables, we need to iteratively find the temperature which gives us the correct density ρ using the p, h equations.

Hence, for steam tables, we need to vary p such that:

$$\rho(p, h) - \rho_{set} = 0$$

By doing so, we will obtain our pressure and (p,h) point.

Now, this step is not done in the EEqn.H step, the algorithm calls for finding the temperature first from the thermodynamic states of ρ and h so that we will get a ρ, T point. This is ideal for putting into a cubic equation of state to find p.

Step 4: pEqn

Now, we enter the PISO part, this is the pressure corrector step. Essentially, it is the momentum equation substituted into the pressure equation.

In the pEqn.H, we see pimpleFoam accounts for transonic equations, but we shall ignore that. We get:

```
if (!pimple.SIMPLErho())
{
    rho = thermo.rho();
}

// Thermodynamic density needs to be updated by psi*d(p) after the
// pressure solution
const volScalarField psip0(psi*p);

volScalarField rAU(1.0/UEqn.A());
surfaceScalarField rhorAUf("rhorAUf", fvc::interpolate(rho*rAU));
volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));

if (pimple.nCorrPISO() <= 1)
{
```

```
    tUEqn.clear();
}

surfaceScalarField phiHbyA
(
    "phiHbyA",
    fvc::interpolate(rho)*fvc::flux(HbyA)
  + MRF.zeroFilter(rhorAUf*fvc::ddtCorr(rho, U, phi, rhoUf))
);

fvc::makeRelative(phiHbyA, rho, U);
MRF.makeRelative(fvc::interpolate(rho), phiHbyA);

// Update the pressure BCs to ensure flux consistency
constrainPressure(p, rho, U, phiHbyA, rhorAUf, MRF);

// NOTE: I'm skipping the transonic part

{
    fvScalarMatrix pDDtEqn
    (
        fvc::ddt(rho) + psi*correction(fvm::ddt(p))
      + fvc::div(phiHbyA)
     ==
        fvOptions(psi, p, rho.name())
    );

    while (pimple.correctNonOrthogonal())
    {
        fvScalarMatrix pEqn(pDDtEqn - fvm::laplacian(rhorAUf, p));

        pEqn.solve(p.select(pimple.finalInnerIter()));

        if (pimple.finalNonOrthogonalIter())
        {
            phi = phiHbyA + pEqn.flux();
        }
    }
}


// Explicitly relax pressure for momentum corrector
```

```
p.relax();

U = HbyA - rAU*fvc::grad(p);
U.correctBoundaryConditions();
fvOptions.correct(U);
K = 0.5*magSqr(U);

if (pressureControl.limit(p))
{
    p.correctBoundaryConditions();
}

thermo.correctRho(psi*p - psip0, rhoMin, rhoMax) ;

#include "rhoEqn.H"
#include "compressibleContinuityErrs.H"

rho = thermo.rho();

// Correct rhoUf if the mesh is moving
fvc::correctRhoUf(rhoUf, rho, U, phi);

if (thermo.dpdt())
{
    dpdt = fvc::ddt(p);

    if (mesh.moving())
    {
        dpdt -= fvc::div(fvc::meshPhi(rho, U), p);
    }
}
```

We recall that the corrector step was done by substituting the expressions in for momentum balance into the mass conservation equations.

So we have:

$$V\frac{\rho^{t+\Delta t}-\rho^{t}}{\Delta t}+\sum_{j}\dot{m}_{j,face}=0$$

$$\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}|+\rho g A_{XS}\frac{\partial z}{\partial x}+A_{XS}\frac{\partial P}{\partial x}=0$$

Now, we just had intermediate guesses from

- density equation ρ

- the momentum predictor, predicting $\dot{m}$
- energy equation, predicting h

The next few steps use the pressure-velocity coupling steps from the SIMPLE and PISO algorithms, but there is a need to update it for density.

Recall for **incompressible** that the momentum equation can be written as:

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p$$

The pressure equation is done by substituing this equation into the continutity equation:

$$\nabla \cdot \mathbf{u} = 0$$
$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \right) = 0$$

By doing so, we can have guesses for the pressure in terms of momentum, or velocity guesses:

$$\nabla \cdot \frac{\nabla P/\rho}{a_p} = \nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \right)$$

This was the original equation mentioned in Jasak's thesis [Jasak, 1996].

That is well and good, but now we have another layer of complication. We move from a pressure based solver to a density based solver. Bone outlines this thought process well [Bone, 2020]. Bone then cites Kurganov et al. for coming up with the central schemes (rhoCentralFoam) [Kurganov and Tadmor, 2000].

Bone mentiones rhoSimpleFoam on page 6-7 [Bone, 2020], this is essential for understanding rhoPimpleFoam. Now, in page 3 of the thesis, Bone mentions about steady state compressible equations.

In that regard, continutity equation is similar to SIMPLE algorithms for incompressible flow, except that ρ is added in. Time derivative terms are not accounted for.

For page 7 onwards, Bone writes about the PISO algorithm for compressible flow [Bone, 2020]. This is more relevant to our case. However, Bone does not elaborate on how density is solved for in rhoPimpleFoam. This is also not covered in the rhoCentralFoam part of the thesis. However, the solution procedure is standard, density, momentum, energy, then EOS [Bone, 2020].

Clues from sonicFoam can also be obtained from Canteros [Canteros and Polanskỳ, 2024] as sonicFoam uses the PIMPLE algorithm like rhoPimpleFoam.

Now, anyway, the first thing that is done is to use the Equation of state to find a pressure.

So now, we have these values which are guessed from before:

- From density eqn: $\rho^{predicted}$

- From momentum predictor: $\dot{m}^{predicted}$
- From energy equation: $h^{predicted}$
- From Equation of State: ρ, T from ρ, h

Now we have:

- p from ρ, T (or ρ, h, should not be an issue either)

So far, there is minimal coupling between momentum and mass balance equation. Hence, we are using these two equations to form the pressure equation from before:

$$V\frac{\rho}{\Delta t} + \sum_j \dot{m}_{j,face} = 0$$

The face fluxes will be dealt with using the momentum predictor as before. It is the pressure time derivative term that is slightly more tricky. Now, to deal with this, the equation of state can be brought in. If we deal with some generic equation of state with a compressibility factor z:

$$\rho = \rho(p, T) = \frac{p}{zRT}$$

The term $\frac{1}{zRT}$ is known as ψ in pEqn.H:

$$\rho = \psi p$$

Suppose now that we can form:

$$d\rho = \psi dp + p d\psi$$

The time derivative becomes equation 2.1:

$$\frac{\partial \rho}{\partial t} = \psi \frac{\partial p}{\partial t} + p \frac{\partial \psi}{\partial t} \tag{2.1}$$

Now, as far as we are concerned, we've already dealt with the energy, and the thermodynamic changes do not need to change and such.

Now, in the first parts of the predictor step, we have already accounted for changes in density due to energy (temperature) increase at constant pressure. This is the second term. The p there will be the pressure at the previous timestep.

Now, due to temperature increase, we already accounted for the change by changing density due to temperature

$$\frac{\partial \rho}{\partial t} = \psi \frac{\partial p}{\partial t} + p \frac{\psi^{\text{after EEqn.H}} - \psi^{\text{previous timestep}}}{\Delta t}$$

Now, if we were to convert this back to expressions of density:

$$\frac{\partial \rho}{\partial t} = \psi \frac{\partial p}{\partial t} + \frac{\rho^{\text{after EEqn.H}} - \rho^{\text{previous timestep}}}{\Delta t}$$

It would be evident now, based on the thermodynamic derivatives why there are two components of the density change, one due to increased enthalpy, one due to pressure.

$$\frac{\partial \rho}{\partial t} = \psi \frac{p^{\text{next timestep}} - p^{\text{previous timestep}}}{\Delta t} + \frac{\rho^{\text{after EEqn.H}} - \rho^{\text{previous timestep}}}{\Delta t}$$

From this regard, the equations now make physical sense. And it is more evident why there are ρ present in the OpenFOAM rhoPimpleFoam equations. Thus, the pressure equation combines the mass balance, equation of state, and momentum balance, in order to predict the pressure field at the next timestep.

Note that after the pressure equation is solved, the mass conservation equation in the density equation is solved again, and then the density is updated using the equation of state.

Now, iterating over these equations many times makes guessing $\rho^{\text{after EEqn.H}}$ much more accurate and consistent with the pressure changes to be done. This results in equation 2.2:

$$\frac{\partial \rho}{\partial t} = \psi \frac{p^{\text{n}+1} - p^{\text{n}}}{\Delta t} + \frac{\rho^{\text{after EEqn.H}} - \rho^{\text{n}}}{\Delta t} \tag{2.2}$$

With this, we get better estimates for pressure at the next timestep.

relaxation factors

Note that there are explicit relaxation factors applied for pressure in the pEqn.H.

Chapter 3

Translation for rhoPimpleFoam to 1D case

With the background of rhoPimpleFoam.C in mind, let us now consider how we can properly discretise our equations obtained from Modelica.

Let's start with pEqn.H, convert that to a rust equivalent for 1D flows, and work our way out.

pEqn equivalent

We know that the momentum predictor equation in 3D is as follows:

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p}$$

Let's develop a momentum predictor equation. From modelica in equation 1.31 [Casella, Leva, et al., 2003] we have:

$$\frac{\partial \dot{m}}{\partial t} + \frac{f_{fanning} P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

For this, I'd rather have momentum predictor equations in terms of mass balance. Let's start with the time dependent terms:

$$\frac{\dot{m}^{t+\Delta t} - \dot{m}^t}{\Delta t} + \frac{f_{fanning} P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

Of course, in the pEqn, we don't actually predict the next timestep mass flow rate straight away. We actually predict the properties at the next iteration. This we denote as n, and $n+1$.

$$\frac{\dot{m}^{n+1} - \dot{m}^n}{\Delta t} + \frac{f_{fanning} P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

Now, we can move on to the next terms. We notice the momentum advection terms are visibly missing compared to rhoPimpleFoam.C. This is indeed a simplification. Will momentum advection terms be significant? As I googled, AI reported that this is significant in high speed flows. (No citation here, may add later). I would like to add it back to keep things consistent with rhoPimpleFoam.C.

How are we doing it? Let's start with the substantial derivative:

$$\rho \frac{D\mathbf{u}}{Dt} = \rho \frac{\partial \mathbf{u}}{\partial t} + \rho(\mathbf{u} \cdot \nabla)\mathbf{u}$$

Let's change this to 1D:

$$\rho \frac{Du}{Dt} = \rho \frac{\partial u}{\partial t} + \rho u \frac{\partial}{\partial x} u$$

Multiply cross sectional area to both sides:

$$\rho A_{XS} \frac{Du}{Dt} = \rho A_{XS} \frac{\partial u}{\partial t} + \rho A_{XS} u \frac{\partial}{\partial x} u$$

$$\frac{D\dot{m}}{Dt} = \frac{\partial \dot{m}}{\partial t} + u \frac{\partial \dot{m}}{\partial x}$$

$$\frac{D\dot{m}}{Dt} = \frac{\partial \dot{m}}{\partial t} + \frac{\dot{m}}{A_{XS}\rho} \frac{\partial \dot{m}}{\partial x}$$

Hence, adding this back:

$$\frac{\dot{m}^{n+1} - \dot{m}^n}{\Delta t} + \frac{\dot{m}}{A_{XS}\rho} \frac{\partial \dot{m}}{\partial x} + \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}|\dot{m}| + \rho g A_{XS} \frac{\partial z}{\partial x} + A_{XS} \frac{\partial P}{\partial x} = 0$$

Now, for the advection term, it is evident that some discretisation is required because we need to find the mass flowrates at the faces using the divergence theorem.

$$\iiint \nabla \cdot \mathbf{u} dV = \iint \mathbf{u} \cdot d\vec{S}$$

Of course, if we were to include density:

$$\rho \iiint \nabla \cdot \mathbf{u} dV = \rho \iint \mathbf{u} \cdot d\vec{S}$$

Now, in a 1D case:

$$\rho A_{XS} \int \frac{\partial}{\partial x} u dx = \rho \iint u \cdot d\vec{S}$$

If we were to replace with sums:

$$\rho A_{XS} \int \frac{\partial}{\partial x} u dx = \rho \sum_j u_j \cdot \vec{S}_j$$

$$\int \frac{\partial \dot{m}}{\partial x} dx = \rho \sum_j u_j \cdot \vec{S}_j$$

If we were to derive an expression back for $\frac{\partial \dot{m}}{\partial x}$:

$$\int \frac{\partial \dot{m}}{\partial x} dx = \rho \sum_j u_j \cdot \vec{S}_j$$

Differentiating both sides with respect to x and noting a consistent cross sectional area across the control volume:

$$\frac{\partial \dot{m}}{\partial x} = \frac{\partial}{\partial x} \rho \sum_j u_j \cdot \vec{S}_j$$

$$\frac{\partial \dot{m}}{\partial x} = \frac{1}{A_{XS}} \frac{\partial}{\partial x} \sum_j \dot{m}_j \cdot \vec{S}_j$$

Note that the dot product implies a sign convention. For divergence, it measures the rate at which a flux diverges (spreads out). In this case, it means outflow of flux (or mass) from a system. Hence $\vec{S}$ necessarily points outwards from the centroid of the cell.

Now, for mass flowrates at the faces, $\dot{m}$, we need some sort of interpolation scheme to have it in terms of the mass flowrates at the centroids. For velocity related interpolation schemes, upwind is used for stability but suffers from diffusivity issues [Alves, Oliveira, and Pinho, 2003].

Let's consider this:

$$\frac{\partial \dot{m}}{\partial x} = \frac{1}{A_{XS} \Delta x} \sum_j \dot{m}_j \cdot \vec{S}_j$$

Let's do upwind interpolation and assume A_{XS}

$$\frac{\partial \dot{m}}{\partial x} = \frac{1}{A_{XS} \Delta x} \sum_j \dot{m}_j \cdot \vec{S}_j$$

$$\frac{\partial \dot{m}}{\partial x} = \frac{1}{A_{XS} \Delta x} \sum_j \dot{m}_j \cdot \vec{S}_j$$

Note: m_j is flow OUT of the control volume:

$$\frac{\partial \dot{m}}{\partial x} = \frac{\sum_j \dot{m}_{j,out}}{\Delta x}$$

In the end, we simplify to this.

If we take mass flowrate as positive in x direction:

$$\frac{\partial \dot{m}}{\partial x} = \frac{\dot{m}_{x+0.5\Delta x} - \dot{m}_{x-0.5\Delta x}}{\Delta x}$$

If we use linear interpolation:

$$\frac{\partial \dot{m}}{\partial x} = \frac{0.5 * (\dot{m}_{CV+1} + \dot{m}_{CV}) - 0.5(\dot{m}_{CV} + \dot{m}_{CV-1})}{\Delta x}$$

$$\frac{\partial \dot{m}}{\partial x} = \frac{0.5\dot{m}_{CV+1} - 0.5\dot{m}_{CV-1}}{\Delta x}$$

Basically, the CV+1 demarcates the properties in the neigbouring cell in the positive x direction, whereas CV-1 demarcates properties in the neigbouring cell in the negative x direction.

This provides a linear way to estimate the mass flowrates at the faces, and is essentially the same as the central difference approximation.

Upwinding means that the mass flowrate depends on the flow direction.

```

// How can we start with upwinding, let's think of 3 cases

// we will have 3 control volumes, 1, 2 and 3,

// we need to find the mass flowrates at face at the interface
// of 1-2, and face 2-3

// note, positive flow means going from 1 -> 2 -> 3
// and negatve flow means 1 <- 2 <- 3

let m1_flow_kg_per_s = 3.0;
let m2_flow_kg_per_s = 5.0;
let m3_flow_kg_per_s = 4.0;

let length_1_meter = 1.0;
let length_2_meter = 2.0;
let length_3_meter = 1.0;

let m12_face_flow_kg_per_s: f64;
let m12_face_flow_kg_per_s: f64;

if m1_flow_kg_per_s > 0.0 && m2_flow_kg_per_s > 0.0 {
        // case 1: this is upwinding in positive flow scenarios
        m12_face_flow_kg_per_s = m1_flow_kg_per_s;
} else if m1_flow_kg_per_s <= 0.0 && m2_flow_kg_per_s <= 0.0 {
```

```
        // case 2: this is upwinding in negative flow scenarios
        m12_face_flow_kg_per_s = m2_flow_kg_per_s;
} else {
        // case 3: no clear direction, then we use linear approximation
        // using length interpolation

        let total_length = length_1_meter + length_2_meter;
        m12_face_flow_kg_per_s
        = m1_flow_kg_per_s * length_2_meter / total_length
        + m2_flow_kg_per_s * length_1_meter / total_length;
}

// So this is upwinding in cases, but if it is an unclear situation, we
// use Gauss linear
```

Meaning to say, if there is influx into the cell, the mass flowrate at the face is the mass flowrate at the neighbouring cell centroid. If there is flow out of the cell, the mass flowrate at the face is the mass flowrate at cell centroid.

If flow is positive in the x direction:

$$\frac{\partial \dot{m}}{\partial x} = \frac{\dot{m}_{CV} - \dot{m}_{CV-1}}{\Delta x}$$

But if flow is in the negative x direction:

$$\frac{\partial \dot{m}}{\partial x} = \frac{\dot{m}_{CV+1} - \dot{m}_{CV}}{\Delta x}$$

Upwind schemes have to depend on this if else statement in the positive and negative x direction. This is essentially the forward and backward difference approximation. It is first order in nature.

From a programming point of view, the upwind scheme is more slightly more difficult to program as compared to the linear approximation.

So we can add this in:

$$\frac{\dot{m}^{n+1} - \dot{m}^{n}}{\Delta t} + \frac{\dot{m}}{A_{XS}\rho}\frac{\sum_j \dot{m}_j}{\Delta x} + \frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

Where $\sum_j \dot{m}_j$ is sum of mass flowrates out of the cell. In terms of time or iteration discretisation, we can linearise it

$$\frac{\dot{m}^{n+1} - \dot{m}^{n}}{\Delta t} + \frac{\dot{m}^{n}}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^{n+1}}{\Delta x} + \frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

For the friction term, we note that the fanning friction factor depends on $\dot{m}$. This will be based on $\dot{m}$ at the previous iteration.

$$\frac{\dot{m}^{n+1} - \dot{m}^n}{\Delta t} + \frac{\dot{m}^n}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^{n+1}}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}\dot{m}^{n+1}|\dot{m}^n| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

The rest of the terms are in the previous iteration. Pressure should also use a Gauss linear approximation.

$$\frac{\dot{m}^{n+1} - \dot{m}^n}{\Delta t} + \frac{\dot{m}^n}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^{n+1}}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}\dot{m}^{n+1}|\dot{m}^n| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x} = 0$$

Hence, our momentum predictor equation becomes:

$$\begin{aligned}&\frac{\dot{m}^{n+1} - \dot{m}^n}{\Delta t} + \frac{\dot{m}^n}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^{n+1}}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}\dot{m}^{n+1}|\dot{m}^n| \\ &= -\rho g A_{XS}\frac{\partial z}{\partial x} - A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x}\end{aligned}$$

Now, we note that $\dot{m}_j$ necessarily implies that mass flowrates of neighbouring cells are considered. Thus, solution of this system of equations will become semi implicit as there is a matrix solution to this.

To simplify this, we will do the following:

$$\begin{aligned}&\frac{\dot{m}^{n+1} - \dot{m}^n}{\Delta t} + \frac{\dot{m}^{n+1}}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^{n}}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}\dot{m}^{n+1}|\dot{m}^n| \\ &= -\rho g A_{XS}\frac{\partial z}{\partial x} - A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x}\end{aligned}$$

In doing so, all mass flowrates can be guessed explicitly. Of course, we can do Gauss-Siedel iterations to update the guesses of the mass flowrates. We can do so until the mass flowrates reached some sort of equilibrium. Otherwise, we can just let the iterations continue till some convergence is reached.

Now, for our momentum predictor we start by going back to the momentum predictor in OpenFOAM. We note that the momentum predictor equation is:

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p}$$

So, we need to arrange things like that of the momentum predictor. This means we move the explicit constants to the right hand side:

$$\frac{\dot{m}^{n+1} - \dot{m}^{n}}{\Delta t} + \frac{\dot{m}^{n+1}}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^n}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}\dot{m}^{n+1}|\dot{m}^n|$$
$$= -\rho g A_{XS}\frac{\partial z}{\partial x} - A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x}$$

$$\frac{\dot{m}^{n+1}}{\Delta t} + \frac{\dot{m}^{n+1}}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^n}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}\dot{m}^{n+1}|\dot{m}^n|$$
$$= \frac{\dot{m}^{n}}{\Delta t} - \rho g A_{XS}\frac{\partial z}{\partial x} - A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x}$$

$$\dot{m}^{n+1}\left(\frac{1}{\Delta t} + \frac{1}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^n}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}|\dot{m}^n|\right)$$
$$= \frac{\dot{m}^{n}}{\Delta t} - \rho g A_{XS}\frac{\partial z}{\partial x} - A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x}$$

It is clear from this that H is defined as shown in equation 3.1:

$$H = \frac{\dot{m}^{n}}{\Delta t} - \rho g A_{XS}\frac{\partial z}{\partial x} \tag{3.1}$$

And the equivalent of $\nabla P/\rho$ is defined in equation 3.2:

$$A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x} \tag{3.2}$$

Then, a_P is in equation 3.3:

$$a_P = \frac{1}{\Delta t} + \frac{1}{A_{XS}\rho}\frac{\sum_j \dot{m}_j^n}{\Delta x} + \frac{f_{fanning}^n P_w}{2\rho A_{XS}^2}|\dot{m}^n| \tag{3.3}$$

Now, the momentum predictor is:

$$\dot{m}_p^{n+1} = \frac{H(\dot{m}^n)}{a_p} + \frac{1}{a_p}A_{XS}\frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x} \tag{3.4}$$

Now, as with Modelica, we should have the option to change the time derivative to zero as well as the momentum advection term using some if else statement. In this case, a_P is simplified to:

$$a_P = \frac{f^n_{fanning} P_w}{2\rho A^2_{XS}} |\dot{m}^n|$$

And H is simplified to:

$$H = -\rho g A_{XS} \frac{\partial z}{\partial x}$$

With this, it should be sufficient to obtain a momentum predictor equation, which takes in all the parameters, then spits out the mass flowrates at the next iteration.

Now, in the case of continuity, we need to know $\nabla \cdot \mathbf{u} = 0$.

However, this is definitely not the case in here. We can start from a 3D density equation, in order to arrive at the pressure coupling equation pEqn.H.

$$\frac{\partial \dot{m}}{\partial t} + \sum_j \dot{m}_j = 0$$

Using the same convention, $\sum_j \dot{m}_j$ is the mass flowrate out of the differential control volume. Since volume is constant:

$$V \frac{\partial \rho}{\partial t} + \sum_j \dot{m}_j = 0$$

$$A_{XS} \Delta x \frac{\partial \rho}{\partial t} + \sum_j \dot{m}_j = 0$$

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{1}{\Delta x} \sum_j \dot{m}_j = 0$$

Or if we don't discretise it:

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}}{\partial x} = 0$$

This is exactly the Modelica equations.

Now, we need an expression for $\frac{\partial \dot{m}_j}{\partial x}$. The only way this is possible is from the momentum advection term. Otherwise, it is very hard to have pressure velocity coupling. Note that the point of the pEqn.H is to predict pressures.

$$\frac{\partial \dot{m}}{\partial t} + \frac{\dot{m}}{A_{XS}\rho} \frac{\partial \dot{m}}{\partial x} + \frac{f_{fanning} P_w}{2\rho A^2_{XS}} \dot{m}|\dot{m}| + \rho g A_{XS} \frac{\partial z}{\partial x} + A_{XS} \frac{\partial P}{\partial x} = 0$$

Now, this isn't going to cut it. The better way, just like in the rhoPimpleFoam equations, is to just substitute the momentum predictor equation straight in in equation 3.4:

$$\dot{m}_p^{n+1} = \frac{H(\dot{m}^n)}{a_p} + \frac{1}{a_p} A_{XS} \frac{0.5P_{CV+1}^n - 0.5P_{CV-1}^n}{\Delta x}$$

Now, it is noteworthy here that we are trying to predict the pressure at the next iteration. So we need to change the notation slightly.

$$\dot{m}_p^{n+1} = \frac{H(\dot{m}^n)}{a_p} + \frac{1}{a_p} A_{XS} \frac{0.5P_{CV+1}^{n+1} - 0.5P_{CV-1}^{n+1}}{\Delta x}$$

We can even go as far to say that the mass flowrate we use for H is based on the momentum predictor:

$$\dot{m}_p^{n+1} = \frac{H(\dot{m}^{predictor})}{a_p} + \frac{1}{a_p} A_{XS} \frac{0.5P_{CV+1}^{n+1} - 0.5P_{CV-1}^{n+1}}{\Delta x}$$

Let's substitute this in the mass balance:

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}}{\partial x} = 0$$

We specifically note that the mass flowrate $\dot{m}$ must be at the next iteration.

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}^{n+1}}{\partial x} = 0$$

To yield:

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \left(\frac{H(\dot{m}^{predictor})}{a_p} + \frac{1}{a_p} A_{XS} \frac{0.5P_{CV+1}^{n+1} - 0.5P_{CV-1}^{n+1}}{\Delta x} \right)}{\partial x} = 0$$

We note that these properties must be evaluated at the faces. Now, the H terms at the faces can be evaluated using the upwind schemes, as well as the a_P terms. This will help in interpolating the terms at the faces. For pressure, we can use a Gauss linear scheme which is second order accurate.

Now, we still must consider the time derivative of density.

We note that density is a function of both temperature and pressure. Recall equation 2.1:

$$\frac{\partial \rho}{\partial t} = \psi \frac{\partial p}{\partial t} + p \frac{\partial \psi}{\partial t}$$

Recall, that we discretise this to equation 2.2:

$$\frac{\partial \rho}{\partial t} = \psi \frac{p^{\mathrm{n}+1} - p^{\mathrm{n}}}{\Delta t} + \frac{\rho^{\text{after EEqn.H}} - \rho^{\mathrm{n}}}{\Delta t}$$

From a coding standpoint, we can start having UEqn.H or its equivalent.

Now, in rust, to keep the nomenclature as similar as possible to OpenFOAM, we can set UEqn as a struct.

Part V

Appendix: Incompressible Equations

To understand rhoPimpleFoam, one should first understand the pressure equations provided by the OpenFOAM CFD manual [Greenshields and Weller, 2022] in Chapter 2.

For incompressible flow, we refer to Pressure Equation.

Equation 3.5 is the incompressible continuity equation:

$$\nabla \bullet \mathbf{u} = 0 \tag{3.5}$$

For newtonian fluid (constant dynamic viscosity and density)

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \bullet (\mathbf{u}\mathbf{u}) = -\nabla p + \mathbf{b} + \nu \nabla^2 \mathbf{u} \tag{3.6}$$

How to get pressure equation? We take divergence of Equation 3.6:

$$\nabla \bullet \frac{\partial \mathbf{u}}{\partial t} + \nabla \bullet \nabla \bullet (\mathbf{u}\mathbf{u}) = -\nabla \bullet \nabla p + \nabla \bullet \mathbf{b} + \nabla \bullet (\nu \nabla^2 \mathbf{u})$$

We use Clairaut's Theorem which states that order of partial derivatives doesn't matter.

$$\frac{\partial \nabla \bullet \mathbf{u}}{\partial t} + \nabla \bullet \nabla \bullet (\mathbf{u}\mathbf{u}) = -\nabla \bullet \nabla p + \nabla \bullet \mathbf{b} + \nabla \bullet (\nu \nabla^2 \mathbf{u})$$

Since $\nabla \bullet \mathbf{u} = 0$,

$$\nabla \bullet \nabla \bullet (\mathbf{u}\mathbf{u}) = -\nabla^2 p + \nabla \bullet \mathbf{b} + \nabla \bullet (\nu \nabla^2 \mathbf{u})$$

Presuming $\mathbf{b}$ is constant,

$$\nabla \bullet \nabla \bullet (\mathbf{u}\mathbf{u}) = -\nabla^2 p + \nabla \bullet (\nu \nabla^2 \mathbf{u})$$

Again, ν is constant, so:

$$\nabla \bullet \nabla \bullet (\mathbf{u}\mathbf{u}) = -\nabla^2 p + \nu(\nabla^2(\nabla \bullet \mathbf{u}))$$

Since $\nabla \bullet \mathbf{u} = 0$,

$$\nabla \bullet [\nabla \bullet (\mathbf{u}\mathbf{u})] = -\nabla^2 p$$

Hence, we have our pressure equation shown in here:

$$\nabla \bullet [\nabla \bullet (\mathbf{u}\mathbf{u})] + \nabla^2 p = 0$$

Bibliography

Aasen, Ailo et al. (2020). "Accurate quantum-corrected cubic equations of state for helium, neon, hydrogen, deuterium and their mixtures". In: *Fluid Phase Equilibria* 524, p. 112790.

Todreas, Neil E, Mujid S Kazimi, and Mahmoud Massoud (2021). *Nuclear systems Volume II: Elements of thermal hydraulic design*. CRC Press.

Malicki, M et al. (2019). "Development of MELCOR thermal hydraulic model of AP1000 and its verification for a DECL break". In: *Annals of Nuclear Energy* 128, pp. 44–52.

Kim, Tae Woon et al. (2014). "Sensitivity study on severe accident core melt progression for advanced PWR using MELCOR code". In: *Nuclear Engineering and Design* 269, pp. 155–159.

Gauntt, RO et al. (2000). "MELCOR computer code manuals". In: *Sandia National Laboratories, NUREG/CR* 6119, p. 785.

Devolder, Brecht et al. (2015). "A review of the implicit motion solver algorithm in OpenFOAM® to simulate a heaving buoy". In: *18th Numerical Towing Tank Symposium (NuTTS'15)*.

Greenshields, Christopher J and Henry G Weller (2022). "Notes on computational fluid dynamics: General principles". In: *(No Title)*.

Penttinen, Olle, Ehsan Yasari, and Håkan Nilsson (2011). "A pimplefoam tutorial for channel flow, with respect to different LES models". In: *Practice Periodical on Structural Design and Construction* 23.2, pp. 1–23.

Jasak, Hrvoje (1996). "Error analysis and estimation in the Finite Volume method with applications to fluid flows". In.

Zimmer, Gerta (2008). "Modelling and simulation of steam turbine processes: individual models for individual tasks". In: *Mathematical and Computer Modelling of Dynamical Systems* 14.6, pp. 469–493.

Kareem, Buliaminu et al. (2018). "Design of steam turbine for electric power production using heat energy from palm kernel shell". In: *Journal of Power and Energy Engineering* 6.11, pp. 111–125.

Dixon, S Larry and Cesare Hall (2013). *Fluid mechanics and thermodynamics of turbomachinery*. Butterworth-Heinemann.

Agromayor, Roberto and Lars O Nord (2019). "Preliminary design and optimization of axial turbines accounting for diffuser performance". In: *International Journal of Turbomachinery, Propulsion and Power* 4.3, p. 32.

Zhang, Haiyan et al. (2022). "Research on the influence factors of brake regenerative energy of pure electric vehicles based on the CLTC". In: *Energy Reports* 8, pp. 85–93.

Cengel, Yunus A and Michael A Boles (2002). "Thermodynamics: an engineering approach". In: *Sea* 1000.8862, pp. 287–293.

Jasak, Hrvoje, Aleksandar Jemcov, Zeljko Tukovic, et al. (2007). "OpenFOAM: A C++ library for complex physics simulations". In: *International workshop on coupled methods in numerical dynamics*. Vol. 1000. Dubrovnik, Croatia), pp. 1–20.

Ong, Theodore Kay Chen (2024). *Digital Twins as Testbeds for Iterative Simulated Neutronics Feedback Controller Development*. University of California, Berkeley.

Casella, Francesco, Alberto Leva, et al. (2003). "Modelica open library for power plant simulation: design and experimental validation". In: *Proceeding of the 2003 Modelica conference, Linkoping, Sweden*. Citeseer.

Bone, Sean (2020). "Comparative study of density-based versus pressure-based solvers for supersonic flow". In: *Eidgenössische Technische Hochschule Zürich Swiss Federal Institute of Technology Zurich*.

Kurganov, Alexander and Eitan Tadmor (2000). "New high-resolution central schemes for nonlinear conservation laws and convection–diffusion equations". In: *Journal of computational physics* 160.1, pp. 241–282.

Canteros, Maria Laura and Jiří Polanskỳ (2024). "Review and comparison of two OpenFOAM® solvers: rhoCentralFoam and sonicFoam". In: *EPJ Web of Conferences*. Vol. 299. EDP Sciences, p. 01005.

Alves, MA, PJ Oliveira, and FT Pinho (2003). "A convergent and universally bounded interpolation scheme for the treatment of advection". In: *International journal for numerical methods in fluids* 41.1, pp. 47–75.